

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO BÀI TẬP
ASSIGNMENT 4.2 (New)
GVHD: TRẦN ĐÌNH QUẾ

LỚP CQ 03 – NHÓM 01

Họ và tên: Đỗ Ngọc Lâm

Mã sinh viên: B22DCCN476

Lớp: D22CNPM04

Hà Nội, 09/2025

MỤC LỤC

DANH MỤC HÌNH ẢNH	3
INSTRUCTIONS.....	5
a) Build a dataset with 10.000 persons and 20 features (jobs, age, gender, height, weight....) and store.....	6
b) Show the distribution of the dataset.....	9
c) Using 5 basic ML models and using BMI to classify and predict.....	11
d) Build models CNN, RNN, LST with ≥ 5 layers for classification and prediction problem in overweight, underweight, normal with respect to age, job, gender, area, diet.....	16
e) Compare and evaluate models given with metrics accuracy, MAE, MSE, RMSE.....	23
f) Visualize the results:	24
g) Deploy the best model so that user enters features and output is underweight, overweight or normal.	26
h) Build a BIG knowledge base more 3MB and store and give a guider for health based on this knowledge base.	27
i) Build guide with chat based on your knowledge base and prediction result	30

DANH MỤC HÌNH ẢNH

Hình 1	6
Hình 2	6
Hình 3	7
Hình 4	7
Hình 5	8
Hình 6	9
Hình 7	9
Hình 8	10
Hình 9	10
Hình 10	11
Hình 11	11
Hình 12	13
Hình 13	13
Hình 14	14
Hình 15	14
Hình 16	15
Hình 17	16
Hình 18	16
Hình 19	17
Hình 20	17
Hình 21	18
Hình 22	18
Hình 23	22
Hình 24	23
Hình 25	23
Hình 26	24
Hình 27	24
Hình 28	25
Hình 29	26
Hình 30	26
Hình 31	27
Hình 32	27
Hình 33	28
Hình 34	28
Hình 35	30
Hình 36	30
Hình 37	31
Hình 38	31

Hình 39	32
Hình 40	32
Hình 41	33
Hình 42	33
Hình 43	34
Hình 44	34

INSTRUCTIONS

Assignment 4.2: TENSORFLOW

Submit file PDF: assign4.2_lớp.nhóm_tênho.pdf

For example: assign4.2_02clc.03_quetd.pdf, assign4.2_02cq.03_quetd.pdf

• Submit before MONDAY (15/09) 11:30PM for both CQ & CLC classes

//same as assignment3

SINH VIÊN CODE VÀ VIẾT GIẢI THÍCH CODE CHI TIẾT

Improve 4.1

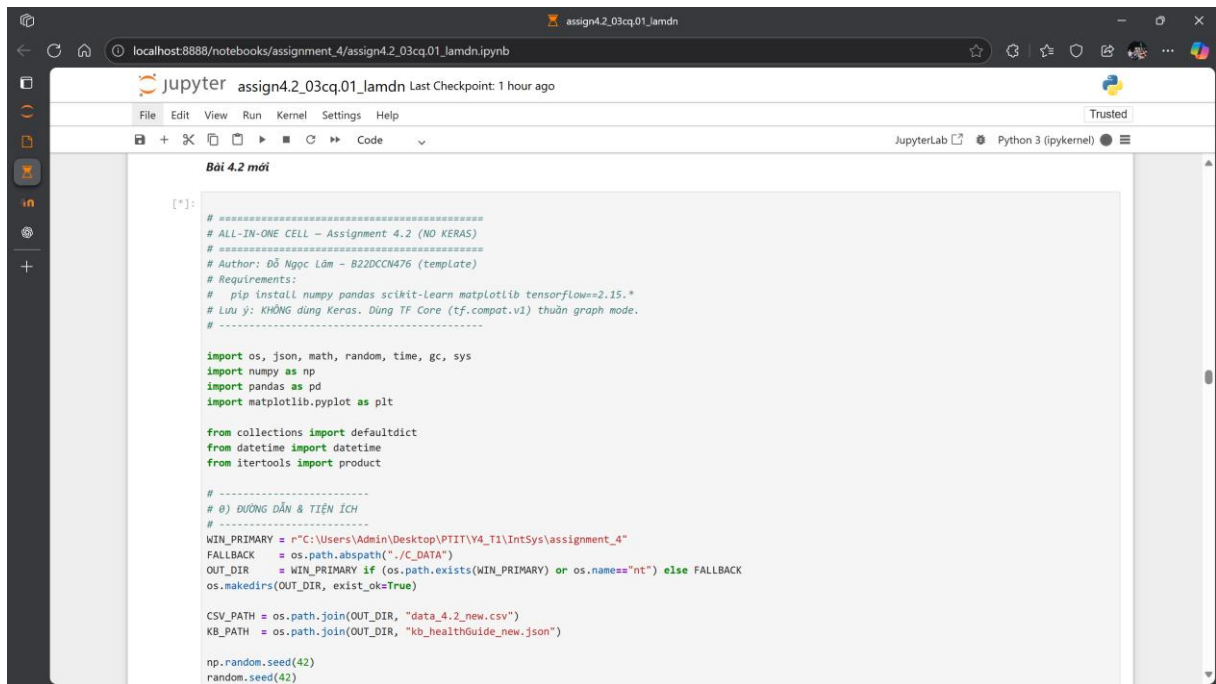
4.2. Extend Assignment 4.1

- Build a dataset with 10.000 persons and 20 features (jobs, age, gender, height, weight....) and store in C:\DATA\data_4.2.csv
- Show the distribution of the dataset
- Using 5 basic ML models and using BMI to classifying and predicting
- Build models CNN, RNN, LSTM with ≥ 5 layers for classification and prediction problem in overweight, underweight, normal with respect to *age, job, gender, area, diet*
- Compare and evaluate models given with metrics accuracy, MAE, MSE, RMSE
- Visualize the results:
 - Which age has more overweight persons?
 - Which job has more overweight persons?
 - Which gender has more overweight persons?
 - Which area has more overweight persons?
- Deploy the best model so that user enters features and output is underweight, overweight or normal
- Build a BIG Knowledge base more 3 M and store in C:\DATA\kb_healthGuide.json and give a guide for health based on this knowledge base.
- Build guide with chat based on your knowledge base and prediction result

Các em cần viết giải thích theo CÁCH HIỂU CỦA MÌNH về:

- Cấu trúc CNN, RNN, LSTM khác nhau thế nào? Batch, Learning rate là gì? Thay đổi learning rate, Batch, epoch...thì kết quả sẽ thế nào?
- data đưa vào cho CNN, RNN, LSTM sẽ biến đổi thế nào reshape()
-Giải thích đầy đủ hơn các biểu đồ
- KB CHƯA TO và output guide chưa thấy!

a) Build a dataset with 10.000 persons and 20 features (jobs, age, gender, height, weight....) and store.



```

[*]:
# *****
# ALL-IN-ONE CELL – Assignment 4.2 (NO KERAS)
# *****
# Author: Đỗ Ngọc Lâm – B22DCCN476 (template)
# Requirements:
# pip install numpy pandas scikit-learn matplotlib tensorflow=2.15.*
# Lưu ý: KHÔNG dùng Keras. Dùng TF Core (tf.compat.v1) thuần graph mode.
# -----

import os, json, math, random, time, gc, sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from collections import defaultdict
from datetime import datetime
from itertools import product

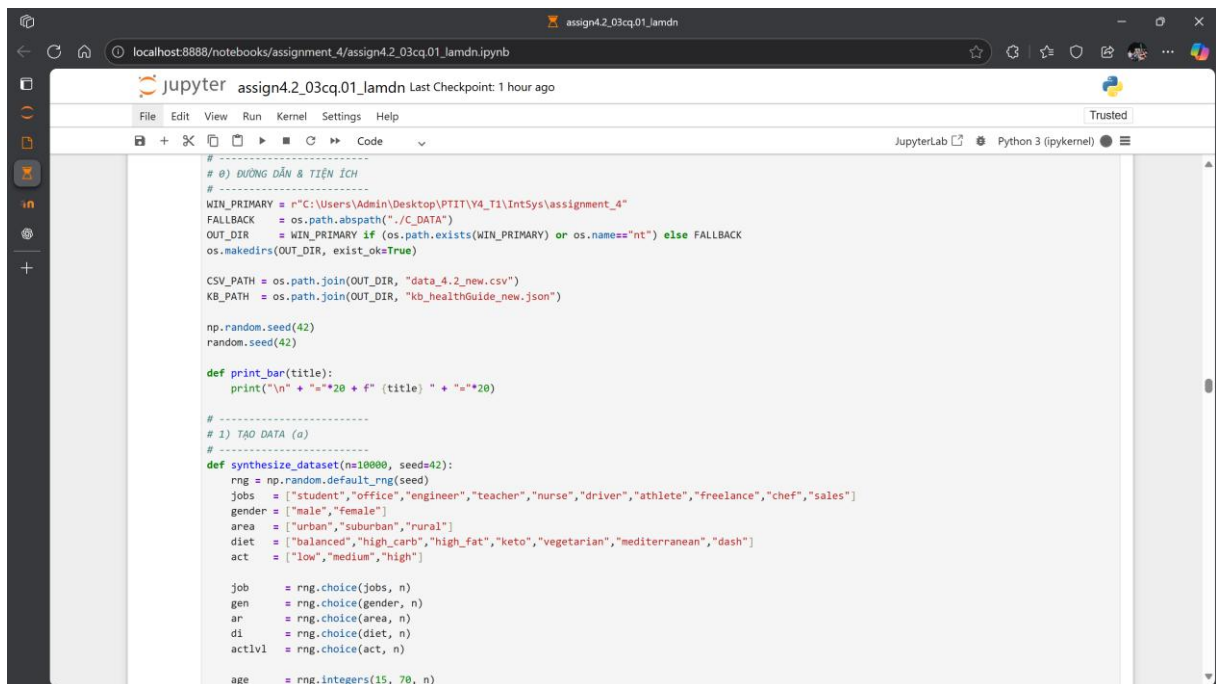
# -----
# 0) ĐƯỜNG DẪN & TIỆN ÍCH
# -----
WIN_PRIMARY = r"C:\Users\Admin\Desktop\PTIT\V4_T1\IntSys\assignment_4"
FALLBACK = os.path.abspath("./C_DATA")
OUT_DIR = WIN_PRIMARY if (os.path.exists(WIN_PRIMARY) or os.name=="nt") else FALLBACK
os.makedirs(OUT_DIR, exist_ok=True)

CSV_PATH = os.path.join(OUT_DIR, "data_4.2_new.csv")
KB_PATH = os.path.join(OUT_DIR, "kb_healthGuide_new.json")

np.random.seed(42)
random.seed(42)

```

Hình 1



```

# -----
# 0) ĐƯỜNG DẪN & TIỆN ÍCH
# -----
WIN_PRIMARY = r"C:\Users\Admin\Desktop\PTIT\V4_T1\IntSys\assignment_4"
FALLBACK = os.path.abspath("./C_DATA")
OUT_DIR = WIN_PRIMARY if (os.path.exists(WIN_PRIMARY) or os.name=="nt") else FALLBACK
os.makedirs(OUT_DIR, exist_ok=True)

CSV_PATH = os.path.join(OUT_DIR, "data_4.2_new.csv")
KB_PATH = os.path.join(OUT_DIR, "kb_healthGuide_new.json")

np.random.seed(42)
random.seed(42)

def print_bar(title):
    print("\n" + "="*20 + f" {title} " + "="*20)

# -----
# 1) TẠO DATA (a)
# -----
def synthesize_dataset(n=10000, seed=42):
    rng = np.random.default_rng(seed)
    jobs = ["student", "office", "engineer", "teacher", "nurse", "driver", "athlete", "freelance", "chef", "sales"]
    gender = ["male", "female"]
    area = ["urban", "suburban", "rural"]
    diet = ["balanced", "high_carb", "high_fat", "keto", "vegetarian", "mediterranean", "dash"]
    act = ["low", "medium", "high"]

    job = rng.choice(jobs, n)
    gen = rng.choice(gender, n)
    ar = rng.choice(area, n)
    di = rng.choice(diet, n)
    activl = rng.choice(act, n)

    age = rng.integers(15, 70, n)

```

Hình 2

```

base_w = 50 + 0.4*(age-18) + rng.normal(0, 6, n)
base_w += np.where(activl=="low", 6, np.where(activl=="high",-4, 0))
base_w += np.where(diet=="high_carb", 3, 0)
base_w += np.where(diet=="keto",-2, 0)
weight_kg = np.clip(base_w, 38, 130)

sugar_g = (rng.normal(60, 20, n)
            + np.where(diet=="high_carb", 25, 0)
            - np.where(diet=="keto", 25, 0)).clip(5, 160)

fat_g = (rng.normal(60, 20, n)
         + np.where(diet=="high_fat", 30, 0)
         + np.where(diet=="keto", 20, 0)).clip(10, 160)

protein_g = (rng.normal(60, 15, n)
             + np.where(activl=="high", 15, 0)
             + np.where(diet=="keto", 10, 0)).clip(20, 180)

# FIX: dùng np.isin thay vì di.isin(...)
fiber_g = (rng.normal(18, 6, n)
          + np.where(np.isin(di, ["vegetarian","mediterranean","dash"]), 5, 0)).clip(5, 50)

waist_cm = (rng.normal(0.45*height_cm, 6, n) + np.where(activl=="low", 4, 0)).clip(55, 140)
bp_sys = rng.normal(118, 15, n) + np.where(age>45, 6, 0) + np.where(diet=="dash",-3, 0)
bp_dia = rng.normal(76, 10, n) + np.where(age>45, 3, 0) + np.where(diet=="dash",-2, 0)
chol = rng.normal(190, 35, n) + np.where(diet=="mediterranean",-10, 0) + np.where(activl=="high",-8,0)

smoker = rng.integers(0, 2, n)
alcohol = rng.integers(0, 20, n)

bmi = weight_kg / ((height_cm/100.0)**2)
def bmi_label(x):
    if x < 18.5: return "under"
    if x < 25.0: return "normal"
    return "over"
y_cls = np.array([bmi_label(x) for x in bmi])

```

Hình 3

```

bmi = weight_kg / ((height_cm/100.0)**2)
def bmi_label(x):
    if x < 18.5: return "under"
    if x < 25.0: return "normal"
    return "over"
y_cls = np.array([bmi_label(x) for x in bmi])

df = pd.DataFrame({
    # 20 features (5 cat + 15 num)
    "job":job, "gender":gen, "area":ar, "diet":di, "activity":activl,
    "age":age, "sleep_h":sleep_h, "steps":steps,
    "height_cm":height_cm.round(1), "weight_kg":weight_kg.round(1),
    "sugar_g":sugar_g.round(1), "fat_g":fat_g.round(1),
    "protein_g":protein_g.round(1), "fiber_g":fiber_g.round(1),
    "waist_cm":waist_cm.round(1), "bp_dia":bp_dia.round(1),
    "bp_sys":bp_sys.round(0), "cholesterol":chol.round(0),
    "smoker":smoker, "alcohol_units":alcohol,
    # tham chiếu
    "BMI":bmi.round(2), "label":y_cls
})
return df

if not os.path.exists(CSV_PATH):
    df = synthesize_dataset(10000)
    df.to_csv(CSV_PATH, index=False, encoding="utf-8")
else:
    df = pd.read_csv(CSV_PATH)

print_bar("DATA SHAPE")
print(df.shape)
print(df.head(3))

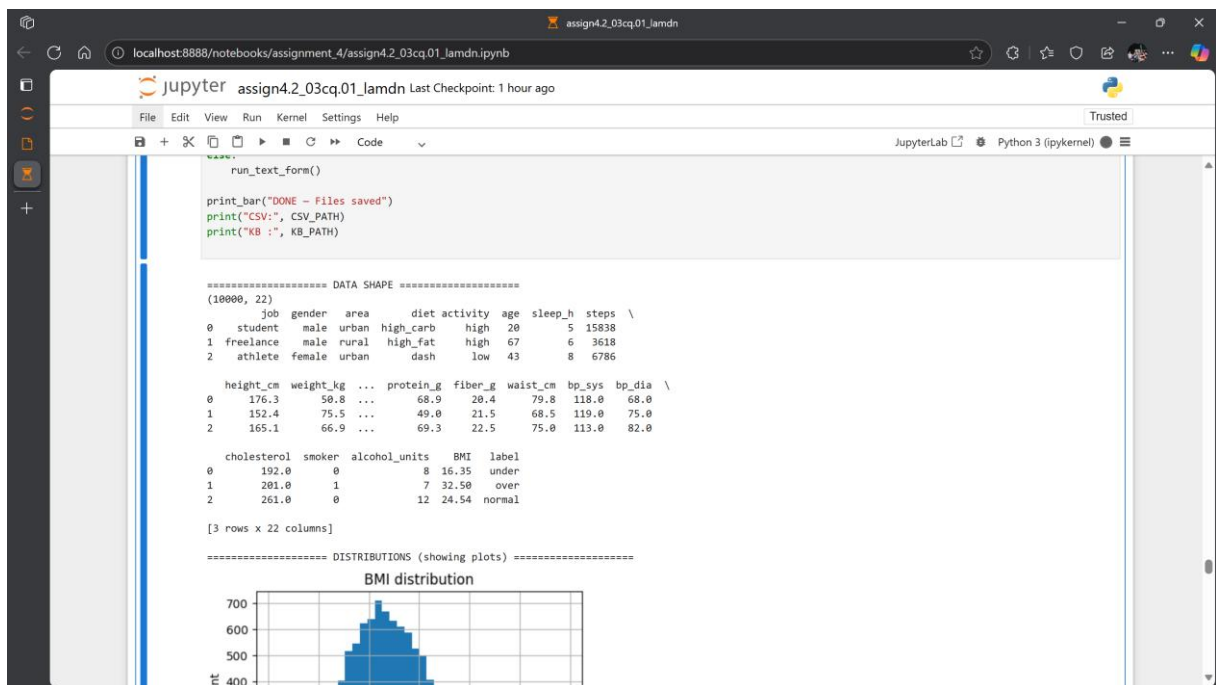
# =====

```

Hình 4

- Import các thư viện cần thiết (ví dụ: os, json, math,...)
- Chuẩn bị các đường dẫn để lưu file data (.csv) và lưu KB (.json)
- Đặt các random_seed
 - ⇒ Ví sao cần phải có:
 - Các hàm random (Python) và np.random (NumPy) có thể ổn định ra một kết quả giống nhau giữa các lần chạy → Dễ tái lập kết quả
- Tạo các danh mục jobs, gender, area, diet, act → đây là 5 cột phân loại chính
- Sau đó chọn ngẫu nhiên từng giá trị cho mỗi người (hàm .choice())

- Thêm các đặc trưng số: age (tuổi), sleep_h (giấc ngủ), steps (bước chân), chiều cao (height), cân nặng, dinh dưỡng → sugarm fat, protein, fiber và các chỉ số sức khỏe khác (waist_cm, bp_sys, bp_dia, colestorel, smoker)
 - Tính BMI và sinh nhân (under, normal, over) → label
 - Kết hợp thành data Frame (20 features = 5 phân loại + 15 số) + 2 cột tham chiếu (BMI và label)
 - Lưu dữ liệu vào CSV_Path
- ⇒ Các hàm đã dùng:
- .choice(list, n): → lấy ngẫu nhiên n phần tử từ list
 - .integers(low, high, n) → sinh n số nguyên trong khoảng [low, high)
 - .normal(mean, std, n) → sinh n số thực theo phân phối chuẩn với trung bình mean, độ lệch chuẩn std
 - .clip(min, max) → ép giá trị nằm trong khoảng [min, max]



Hình 5

b) Show the distribution of the dataset

```
print(df.head(3))

# -----
# 2) PHÂN PHỐI (b)
# -----
print_bar("DISTRIBUTIONS (showing plots)")
plt.figure(figsize=(5,3)); df["BMI"].hist(bins=40); plt.title("BMI distribution"); plt.xlabel("BMI"); plt.ylabel("count"); plt.show()
plt.figure(figsize=(5,3)); df["age"].hist(bins=30); plt.title("Age distribution"); plt.xlabel("age"); plt.ylabel("count"); plt.show()
plt.figure(figsize=(5,3)); df["weight_kg"].hist(bins=40); plt.title("Weight (kg)"); plt.show()
plt.figure(figsize=(5,3)); df["height_cm"].hist(bins=40); plt.title("Height (cm)"); plt.show()
plt.figure(figsize=(5,3)); df["steps"].hist(bins=40); plt.title("Steps/day"); plt.show()

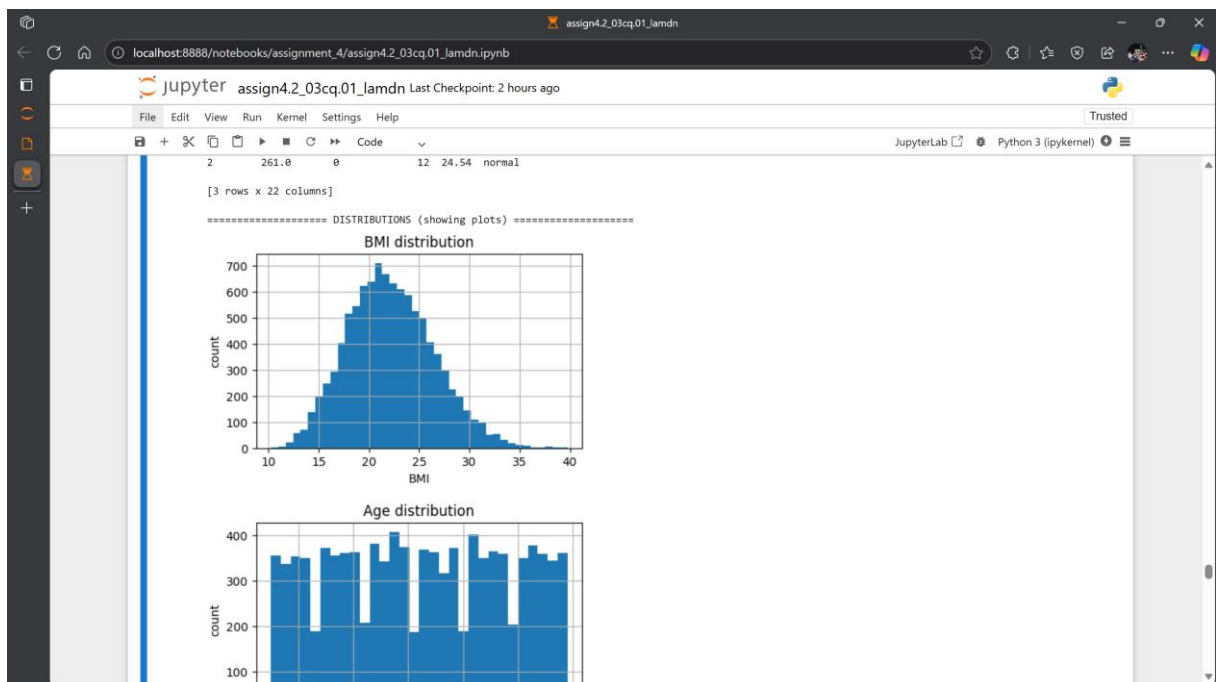
# -----
# 3) TIỀN XỬ LÝ + 5 ML MODELS (c)
# -----
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, mean_absolute_error, mean_squared_error, confusion_matrix, classification_report

from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

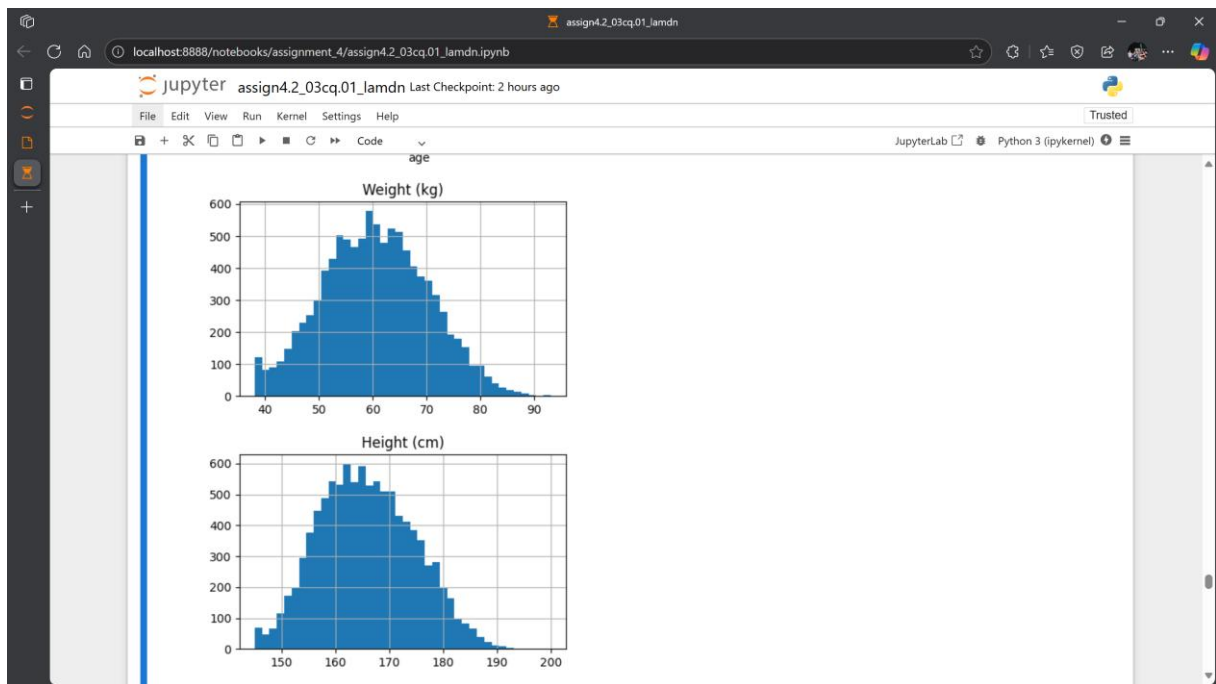
target = "label"
X = df.drop(columns=[target, "BMI"])
# X = df[["age", "job", "gender", "area", "diet"]]
y = df[target]

num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = [c for c in X.columns if c not in num_cols]
```

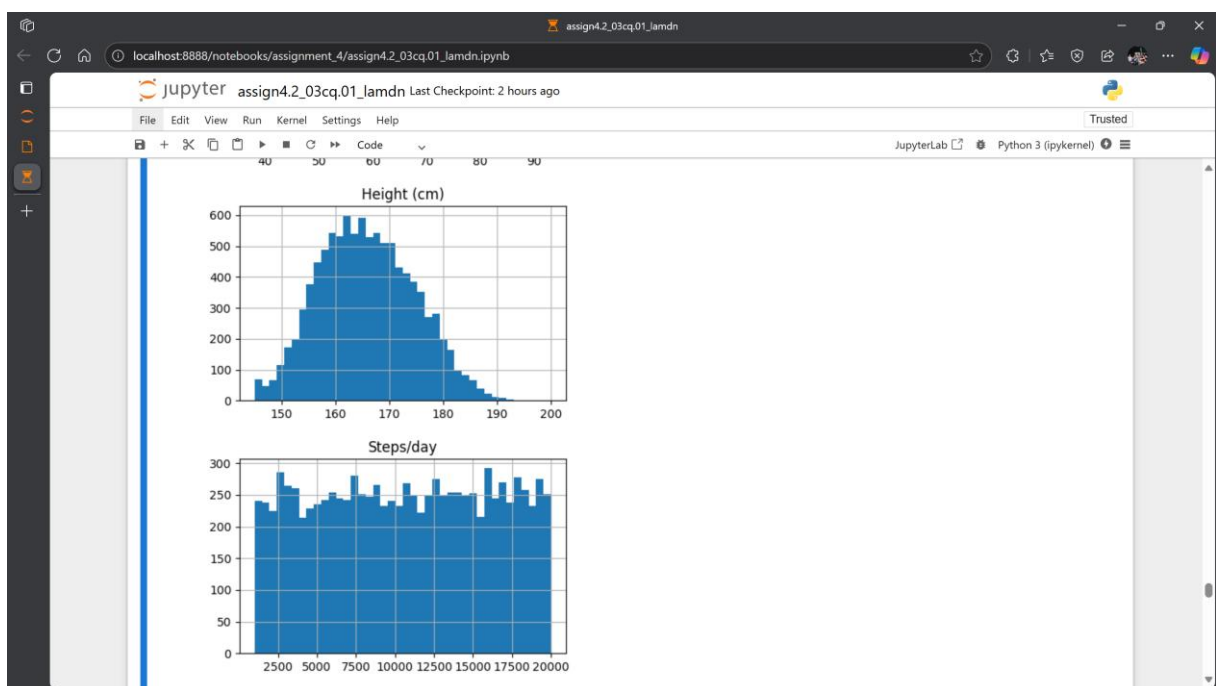
Hình 6



Hình 7



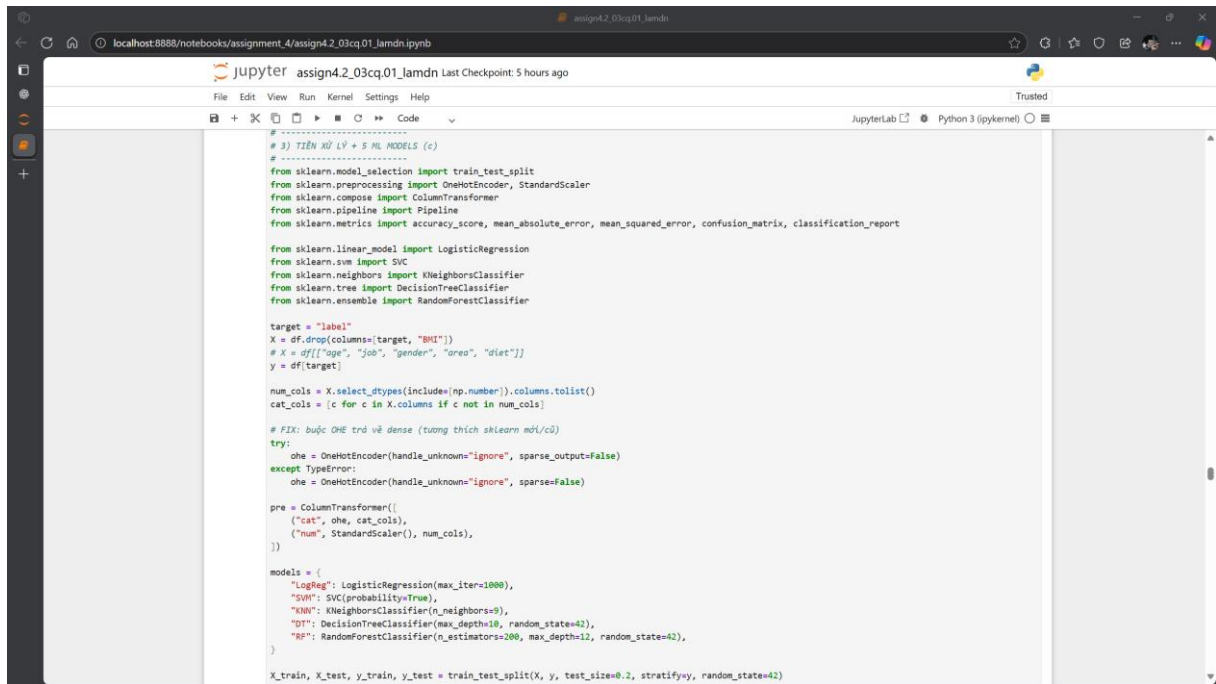
Hình 8



Hình 9

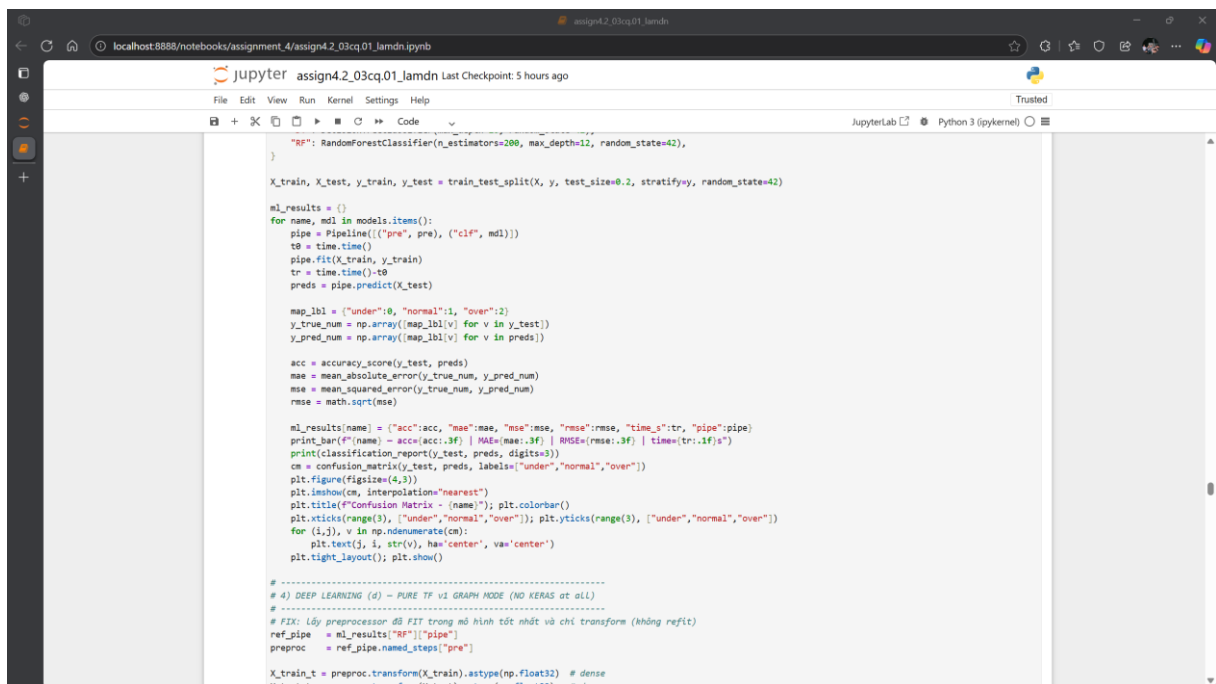
- ➔ Biểu đồ BMI distribution: Đây là biểu đồ cho thấy tỉ lệ người under/normal/overweight. Từ biểu đồ ta thấy phần lớn mọi người ở trong khoảng 18.5 – 25.
- ➔ Biểu đồ Age distribution: Biểu đồ phân bố tuổi
- ➔ Biểu đồ Weight Distribution: Biểu đồ phân bố cân nặng
- ➔ Biểu đồ Height distribution: Biểu đồ phân bố chiều cao
- ➔ Biểu đồ Steps/day: Biểu đồ phân bố số lượng bước đi hàng ngày

c) Using 5 basic ML models and using BMI to classify and predict



```
# ---  
# 3) TIỀN XỬ LÝ + 5 ML MODELS (c)  
# ---  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import OneHotEncoder, StandardScaler  
from sklearn.compose import ColumnTransformer  
from sklearn.pipeline import Pipeline  
from sklearn.metrics import accuracy_score, mean_absolute_error, mean_squared_error, confusion_matrix, classification_report  
  
from sklearn.linear_model import LogisticRegression  
from sklearn.svm import SVC  
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.ensemble import RandomForestClassifier  
  
target = "label"  
X = df.drop(columns=target, "BMI")  
# X = df[["age", "job", "gender", "area", "diet"]]  
y = df[target]  
  
num_cols = X.select_dtypes(include=[np.number]).columns.tolist()  
cat_cols = [c for c in X.columns if c not in num_cols]  
  
# FIX: buộc OHE trả về dense (tuong thích sklearn mdl/cũ)  
try:  
    ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)  
except TypeError:  
    ohe = OneHotEncoder(handle_unknown="ignore", sparse=False)  
  
pre = ColumnTransformer([  
    ("cat", ohe, cat_cols),  
    ("num", StandardScaler(), num_cols),  
)  
  
models = {  
    "LogReg": LogisticRegression(max_iter=1000),  
    "SVM": SVC(probability=True),  
    "KNN": KNeighborsClassifier(n_neighbors=9),  
    "DT": DecisionTreeClassifier(max_depth=10, random_state=42),  
    "RF": RandomForestClassifier(n_estimators=200, max_depth=12, random_state=42),  
}  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
```

Hình 10



```
models["RF"] = RandomForestClassifier(n_estimators=200, max_depth=12, random_state=42),  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)  
  
ml_results = {}  
for name, mdl in models.items():  
    pipe = Pipeline([("pre", pre), ("clf", mdl)])  
    t0 = time.time()  
    pipe.fit(X_train, y_train)  
    tr = time.time() - t0  
    preds = pipe.predict(X_test)  
  
    map_lbl = {"under":0, "normal":1, "over":2}  
    y_true_num = np.array([map_lbl[v] for v in y_test])  
    y_pred_num = np.array([map_lbl[v] for v in preds])  
  
    acc = accuracy_score(y_test, preds)  
    mae = mean_absolute_error(y_true_num, y_pred_num)  
    mse = mean_squared_error(y_true_num, y_pred_num)  
    rmse = math.sqrt(mse)  
  
    ml_results[name] = {"acc":acc, "mae":mae, "mse":mse, "rmse":rmse, "time_s":tr, "pipe":pipe}  
    print_bar(f"{name} - acc:{acc:.3f} | MAE:{mae:.3f} | RMSE:{rmse:.3f} | time:{tr:.1f}s")  
    print(classification_report(y_test, preds, digits=3))  
    cm = confusion_matrix(y_test, preds, labels=["under", "normal", "over"])  
    plt.figure(figsize=(4,3))  
    plt.imshow(cm, interpolation="nearest")  
    plt.title(f"Confusion Matrix - {name}"); plt.colorbar()  
    plt.xticks(range(3), ["under", "normal", "over"]); plt.yticks(range(3), ["under", "normal", "over"])  
    for (i,j), v in np.ndenumerate(cm):  
        plt.text(j, i, str(v), ha="center", va="center")  
    plt.tight_layout(); plt.show()  
  
# ---  
# 4) DEEP LEARNING (d) - PURE TF v1 GRAPH MODE (NO KERAS or GLL)  
# ---  
# FIX: lấy preprocessor để FIT trong mô hình tốt nhất và chỉ transform (không refit)  
ref_pipe = ml_results["RF"][("pipe")]  
preproc = ref_pipe.named_steps["pre"]  
  
X_train_t = preproc.transform(X_train).astype(np.float32) # dense  
X_test_t = preproc.transform(X_test).astype(np.float32) # dense
```

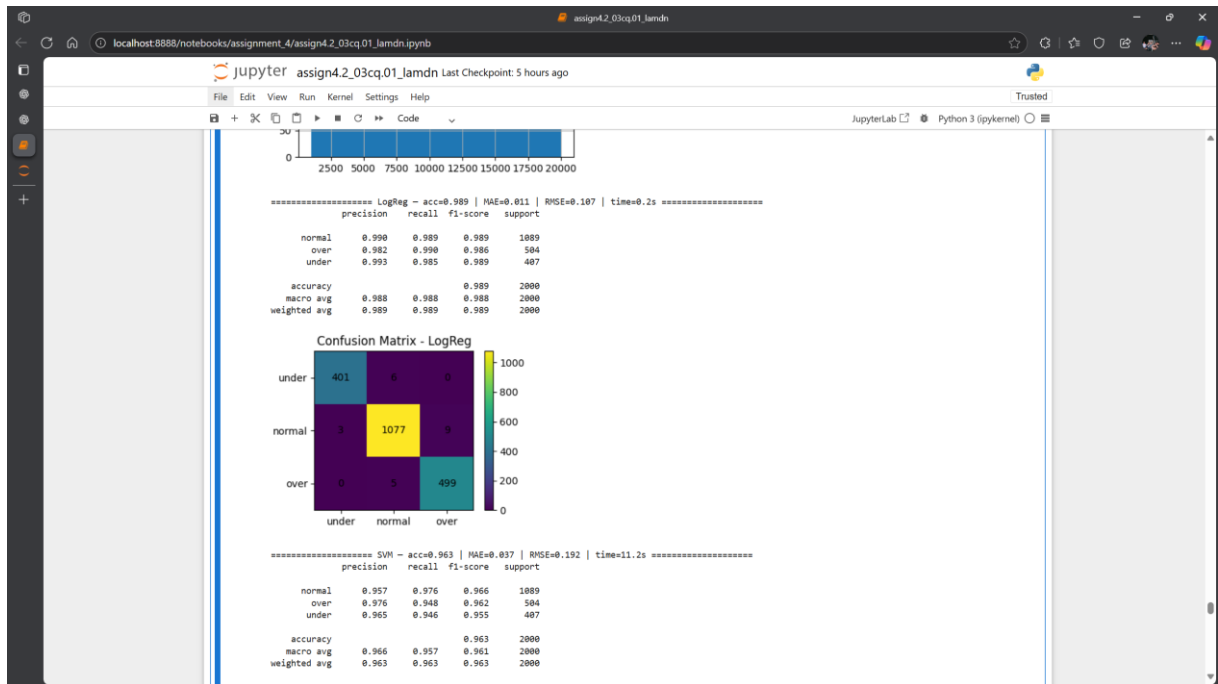
Hình 11

➔ Các bước code

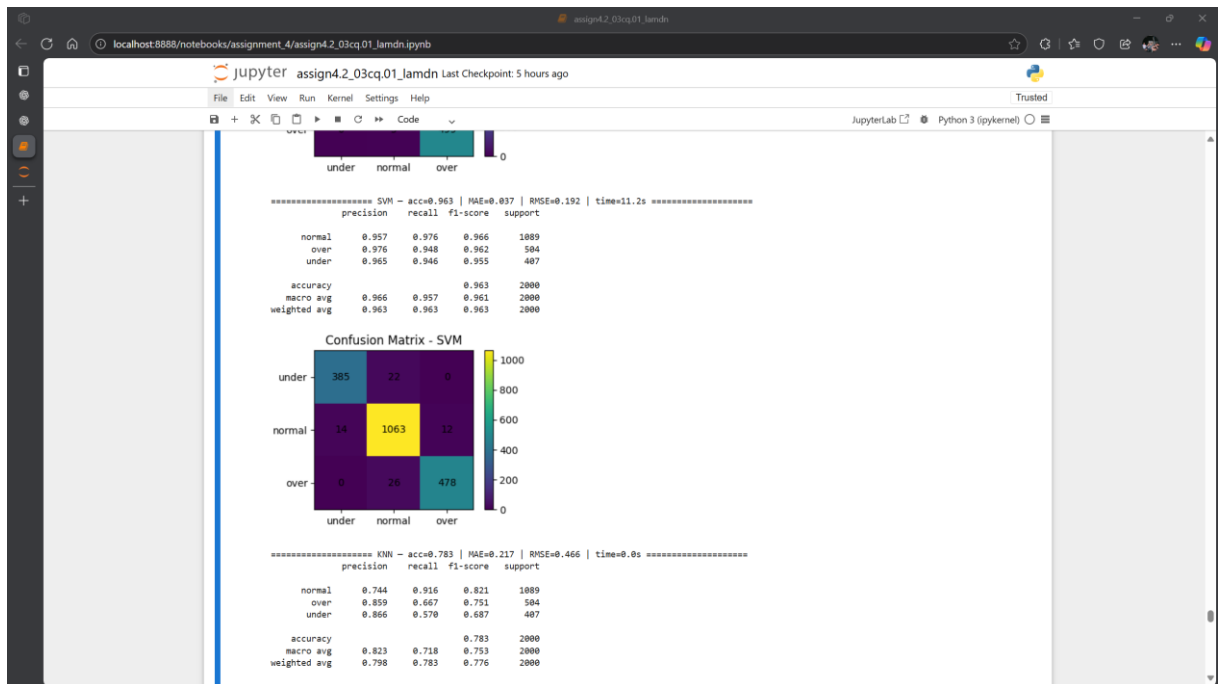
- Import các thư viện cần thiết
- Chuẩn bị data (X là tập features, Y là tập label hay target)
- Chuẩn hóa data ➔ Dùng OneHotEncoder để chuyển các data có dạng khác number về number và dùng StandardScaler() để chuẩn hóa các data, tránh lệch thước đo gây bias mô hình

- Khai báo và cấu hình 5 mô hình : LogReg, SVM, KNN, DT, RF
- Chia data train và test (test_size = 20%)
- Huấn luyện qua Pipeline
- Tính chỉ số và đánh giá kết quả (ACC, MAE, MSE, RMSE)
- ➔ Giới thiệu về các model
- Logistic Regression (LogReg)
 - Bản chất: mô hình tuyến tính học biên phân tách tuyến tính trong không gian đặc trưng, tối ưu log-loss
 - Ưu: Nhanh, dễ huấn luyện, diễn giải tốt, hoạt động ổn nếu quan hệ gần tuyến tính
 - Nhược: Khó bắt quan hệ phi tuyến phức tạp nếu không mở rộng đặc trưng
- Support Vector Classifier (SVM)
 - Bản chất: tìm siêu phẳng tối đa khoảng cách biên
 - Ưu: Mạnh với dữ liệu có biên rõ, ít outlier
 - Nhược: Nhạy cảm scale, khá chậm khi mẫu lớn
- K-Nearest Neighbors (KNN)
 - Bản chất: Dự đoán theo đa số nhãn của k hàng xóm gần nhất
 - Ưu: ý tưởng đơn giản, phi tham số, bắt quan hệ phi tuyến tự nhiên
 - Nhược: Tốc độ dự đoán chậm nếu test lớn, nhạy với nhiễu và thang đo.
- Decision Tree
 - Bản chất: Học cây nhị phân theo tiêu chí giảm impurity, chia theo ngưỡng đặc trưng
 - Ưu: diễn giải dễ, bắt phi tuyến/ tương tác tự nhiên và không cần scale
 - Nhược: Dễ overfit nếu không giới hạn độ sâu
- Random Forest
 - Bản chất: Tập hợp (ensemble) n_estimators cây (bagging + chọn ngẫu nhiên đặc trưng khi split) → giảm phương sai, tổng quát tốt
 - Ưu: Thuônwgf mạnh trên tabular, ít cần tinh chỉnh, kháng nhiễu

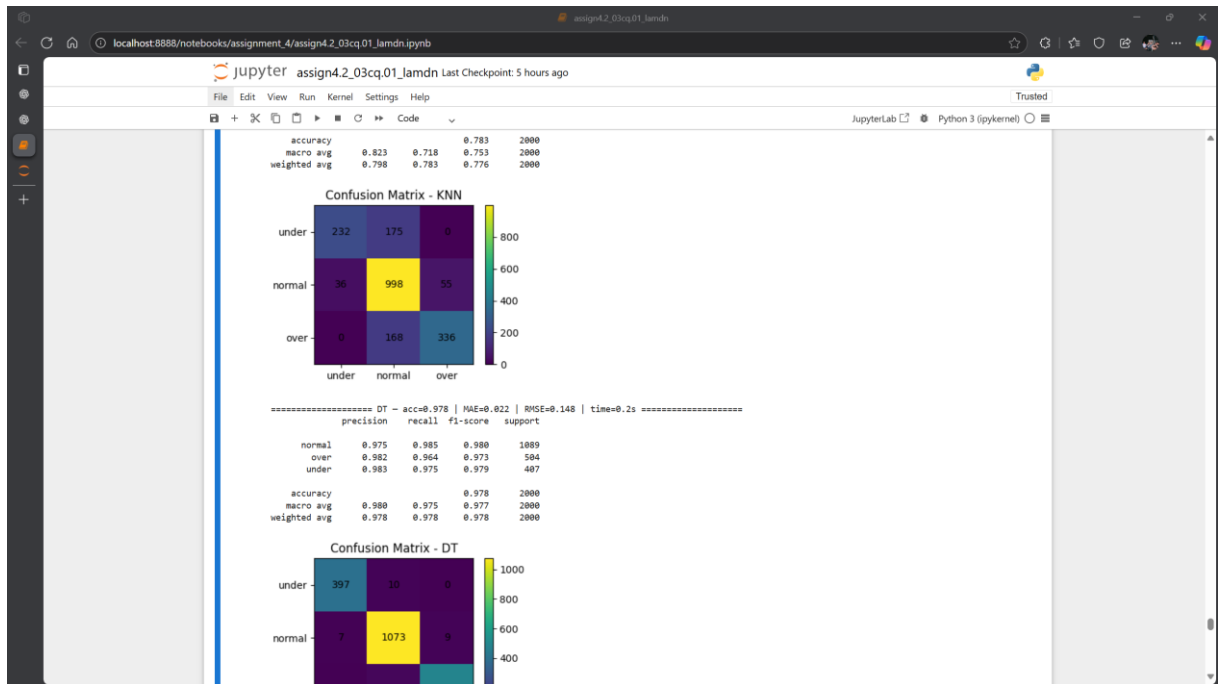
⇒ Kết quả



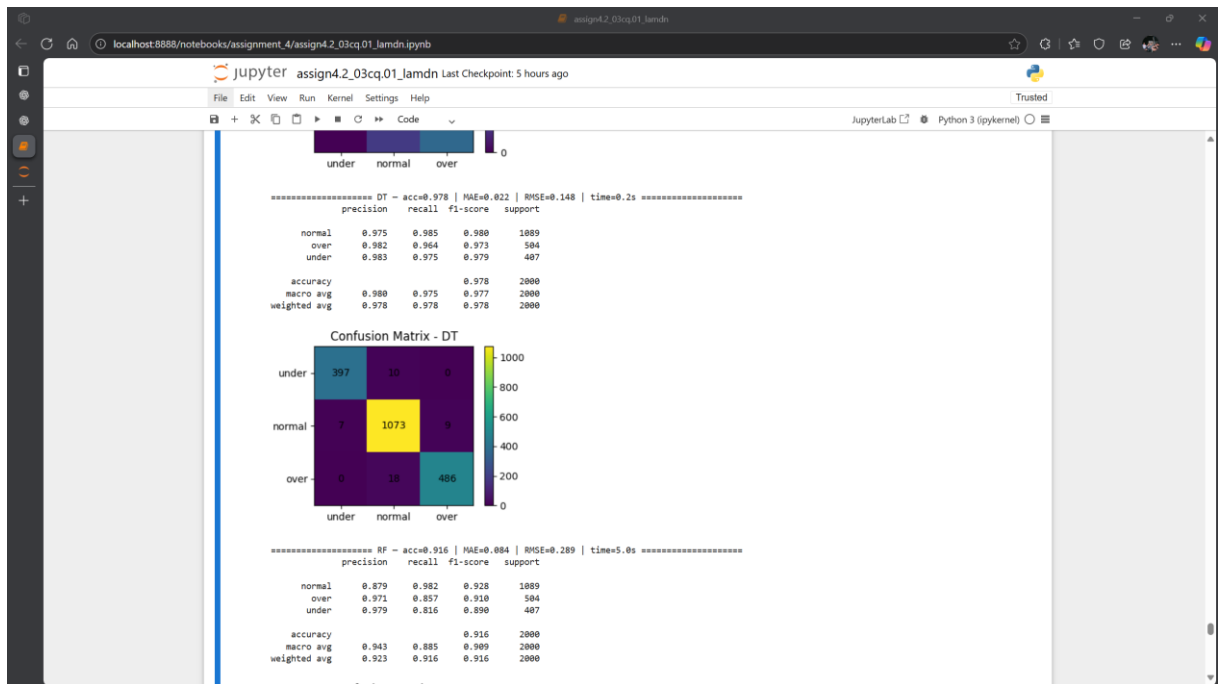
Hình 12



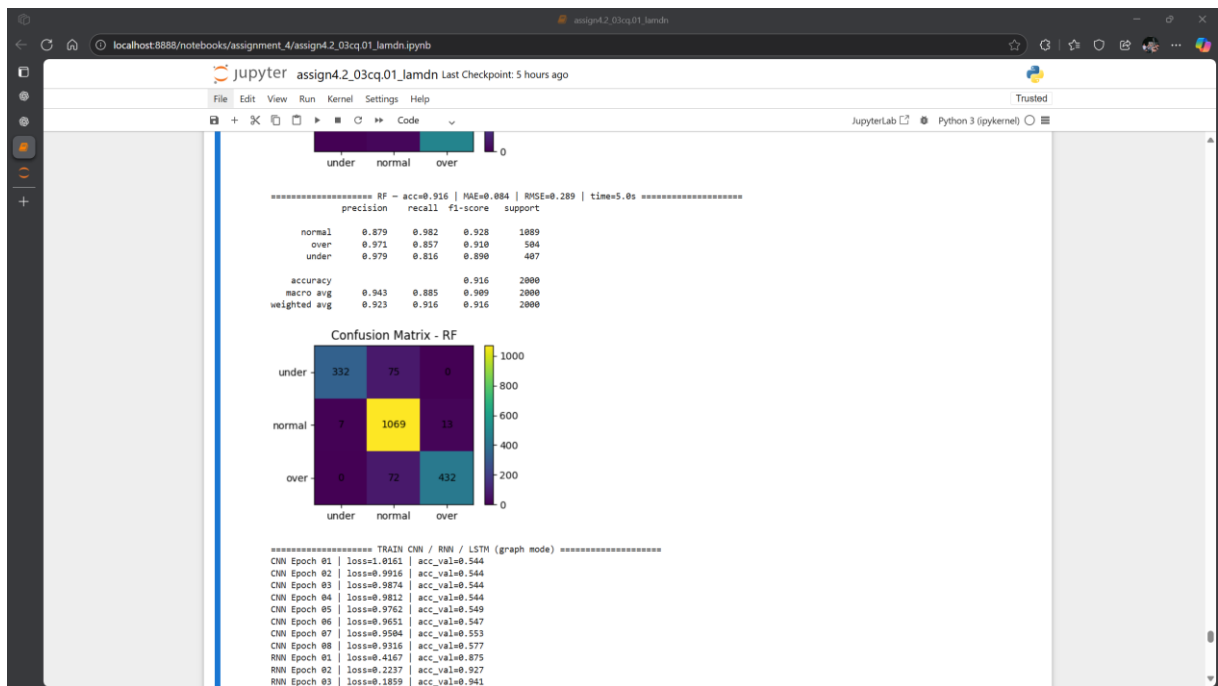
Hình 13



Hình 14



Hình 15



Hình 16

d) Build models CNN, RNN, LST with ≥ 5 layers for classification and prediction problem in overweight, underweight, normal with respect to age, job, gender, area, diet.

```

plt.tight_layout(); plt.show()

# -----
# 4) DEEP LEARNING (d) - PURE TF v1 GRAPH MODE (NO KERAS at all)
# -----
# FIX: Lấy preprocessor đã FIT trong mô hình tốt nhất và chỉ transform (không refit)
ref_pipe = ml_results["ref"]["pipe"]
preproc = ref_pipe.named_steps["pre"]

X_train_t = preproc.transform(X_train).astype(np.float32) # dense
X_test_t = preproc.transform(X_test).astype(np.float32) # dense
feat_dim = X_train_t.shape[1]
label2id = {"under":0, "normal":1, "over":2}
y_train_id = np.array([label2id[v] for v in y_train], dtype=np.int32)
y_test_id = np.array([label2id[v] for v in y_test], dtype=np.int32)

import tensorflow as tf
tf.compat.v1.disable_eager_execution()
tf.random.set_seed(42)

num_classes = 3
hidden_size = 64
num_layers = 5

# ---- 4.1 CNN-1D (graph mode) ----
g_cnn = tf.Graph()
with g_cnn.as_default():
    Xc = tf.compat.v1.placeholder(tf.float32, shape=[None, feat_dim], name="X")
    yc = tf.compat.v1.placeholder(tf.int32, shape=[None], name="y")
    x1 = tf.expand_dims(Xc, -1) # [B, T=feat_dim, C=1]

    Wc1 = tf.Variable(tf.random.normal([5, 1, 32], stddev=0.1))
    Wc2 = tf.Variable(tf.random.normal([5, 32, 64], stddev=0.1))
    Wc3 = tf.Variable(tf.random.normal([3, 64, 64], stddev=0.1))
    z = tf.nn.conv2d(x1, Wc1, [1, 1, 1, 1], padding="SAME")
    z = tf.nn.conv2d(z, Wc2, [1, 1, 1, 1], padding="SAME")
    z = tf.nn.conv2d(z, Wc3, [1, 1, 1, 1], padding="SAME")
    gap = tf.reduce_mean(z, axis=[-1, -2]) # [B, 64]

    Wd1 = tf.Variable(tf.random.normal([64, 64], stddev=0.1)); bd1 = tf.Variable(tf.zeros([64]))
    Wd = tf.Variable(tf.random.normal([64, num_classes], stddev=0.1)); bo = tf.Variable(tf.zeros([num_classes]))
    hid = tf.nn.relu(tf.matmul(gap, Wd1) + bd1)
    logits_cnn = tf.matmul(hid, Wd) + bo

```

Hình 17

```

logits_cnn = tf.matmul(hid, Wd) + bo

loss_cnn = tf.reduce_mean(tf.nn.sparse_softmax_cross_entropy_with_logits(labels=yc, logits=logits_cnn))
opt_cnn = tf.compat.v1.train.AdamOptimizer(1e-3).minimize(loss_cnn)
pred_cnn = tf.argmax(logits_cnn, axis=-1)
acc_cnn = tf.reduce_mean(tf.cast(tf.equal(pred_cnn, tf.cast(yc, tf.int64)), tf.float32))
init_cnn = tf.compat.v1.global_variables_initializer()

def train_cnn_graph(epochs=8, batch=128):
    with tf.compat.v1.Session(graph=g_cnn) as sess:
        sess.run(init_cnn)
        n = X_train_t.shape[0]
        for ep in range(1, epochs+1):
            idx = np.random.permutation(n)
            Xp, yp = X_train_t[idx], y_train_id[idx]
            losses = []
            for i in range(0, n, batch):
                xb = Xp[i:i+batch]; yb = yp[i:i+batch]
                _, l = sess.run([opt_cnn, loss_cnn], feed_dict={Xc:xb, yc:yb})
                losses.append(l)
            acc_val = sess.run(acc_cnn, feed_dict={Xc:X_test_t, yc:y_test_id})
            print(f"CNN Epoch (ep:{ep}) | loss={np.mean(losses):.4f} | acc_val={acc_val:.3f}")
        return Float(acc_val)

```

Hình 18


```

losses.append(1)
acc_val = sess.run(acc_cnn, feed_dict={Xc:X_test_t, yc:y_test_id})
print(f"CNN Epoch {ep:02d} | loss={np.mean(losses):.4f} | acc_val={acc_val:.3f}")
return float(acc_val)

# ---- 4.2 RNN (5 Layers) thuần TF ----
def build_stacked_rnn_graph(feats_dim, hidden=64, layers=5, n_classes=3, scope="RNNCore"):
    g = tf.Graph()
    with g.as_default():
        X = tf.compat.v1.placeholder(tf.float32, shape=[None, feats_dim], name="X") # [B, T]
        y = tf.compat.v1.placeholder(tf.int32, shape=[None], name="y")
        x_seq = tf.expand_dims(X, -1) # [B, T, 1]

        params = []
        in_dim = 1
        for l in range(layers):
            with tf.compat.v1.variable_scope(f"scope_{l+1}"):
                W_hh = tf.Variable(tf.random.normal([in_dim, hidden], stddev=0.1), name="W_hh")
                W_hx = tf.Variable(tf.random.normal([hidden, hidden], stddev=0.1), name="W_hx")
                b = tf.Variable(tf.zeros([hidden]), name="b")
                params.append((W_hh, W_hx, b))
                in_dim = hidden

        batch_size_dyn = tf.shape(X)[0]
        h_states = [tf.zeros([batch_size_dyn, hidden], dtype=tf.float32) for _ in range(layers)]

        def step_once(x_t, h_list):
            inp = x_t
            new_h = []
            for (W_hh, W_hx, b), h_prev in zip(params, h_list):
                h = tf.tanh(tf.matmul(inp, W_hh) + tf.matmul(h_prev, W_hx) + b)
                new_h.append(h)
                inp = h
            return new_h

        for t in range(feats_dim):
            x_t = x_seq[:, t, :] # [B, 1]
            h_states = step_once(x_t, h_states)

        last_h = h_states[-1]
        W_o = tf.Variable(tf.random.normal([hidden, n_classes], stddev=0.1), name="W_o")
        b_o = tf.Variable(tf.zeros([n_classes]), name="b_o")
        logits = tf.matmul(last_h, W_o) + b_o

```

Hình 19

```

loss = tf.reduce_mean(tf.nn.sparse_softmax_cross_entropy_with_logits(labels=y, logits=logits))
train_op = tf.compat.v1.train.AdamOptimizer(1e-3).minimize(loss)
preds = tf.argmax(logits, axis=-1)
acc = tf.reduce_mean(tf.cast(tf.equal(preds, tf.cast(y, tf.int64)), tf.float32))
init = tf.compat.v1.global_variables_initializer()
return ("graph":g, "X":X, "y":y, "loss":loss, "train_op":train_op, "acc":acc, "init":init)

# ---- 4.3 LSTM (5 Layers) thuần TF ----
def build_stacked_lstm_graph(feats_dim, hidden=64, layers=5, n_classes=3, scope="LSTMCore"):
    g = tf.Graph()
    with g.as_default():
        X = tf.compat.v1.placeholder(tf.float32, shape=[None, feats_dim], name="X")
        y = tf.compat.v1.placeholder(tf.int32, shape=[None], name="y")
        x_seq = tf.expand_dims(X, -1)

        params = []
        in_dim = 1
        for l in range(layers):
            with tf.compat.v1.variable_scope(f"scope_{l+1}"):
                W_x = tf.Variable(tf.random.normal([in_dim, 4*hidden], stddev=0.1), name="W_x")
                W_h = tf.Variable(tf.random.normal([hidden, 4*hidden], stddev=0.1), name="W_h")
                b = tf.Variable(tf.zeros([4*hidden]), name="b")
                params.append((W_x, W_h, b))
                in_dim = hidden

        batch_size_dyn = tf.shape(X)[0]
        h_states = [tf.zeros([batch_size_dyn, hidden], dtype=tf.float32) for _ in range(layers)]
        c_states = [tf.zeros([batch_size_dyn, hidden], dtype=tf.float32) for _ in range(layers)]

        def sigmoid(z): return 1.0 / (1.0 + tf.exp(-z))

        def step_once(x_t, h_list, c_list):
            inp = x_t
            new_h, new_c = [], []
            for (W_x, W_h, b), h_prev, c_prev in zip(params, h_list, c_list):
                z = tf.matmul(inp, W_x) + tf.matmul(h_prev, W_h) + b # [B, 4h]
                i, f, o, g = tf.split(z, num_or_size_splits=4, axis=-1)
                i = sigmoid(i); f = sigmoid(f); o = sigmoid(o); g = tf.tanh(g)
                c = f * c_prev + i * g
                h = o * tf.tanh(c)
                new_h.append(h); new_c.append(c)
            inp = h
            return new_h, new_c

```

Hình 20

```

c_states = tf.zeros([batch_size_dyn, hidden], dtype=tf.float32) for _ in range(layers))

def sigmoid(z): return 1.0 / (1.0 + tf.exp(-z))

def step_once(x_t, h_list, c_list):
    inp = x_t
    new_h, new_c = [], []
    for (W_h, W_h_b), h_prev, c_prev in zip(params, h_list, c_list):
        z = tf.matmul(inp, W_h) + tf.matmul(h_prev, W_h_b) + b # [B, 4h]
        i, f, o, g = tf.split(z, num_or_size_splits=4, axis=1)
        i = sigmoid(i); f = sigmoid(f); o = sigmoid(o); g = tf.tanh(g)
        c = f * c_prev + i * g
        h = o * tf.tanh(c)
        new_h.append(h); new_c.append(c)
    new_h, new_c = tf.concat(new_h, axis=0), tf.concat(new_c, axis=0)
    return new_h, new_c

for t in range(feast_dim):
    x_t = x_seq[:, t, :]
    h_states, c_states = step_once(x_t, h_states, c_states)

last_h = h_states[-1]
Wo = tf.Variable(tf.random_normal([hidden, n_classes], stddev=0.1), name="Wo")
bo = tf.Variable(tf.zeros([n_classes]), name="bo")
logits = tf.matmul(last_h, Wo) + bo

loss = tf.reduce_mean(tf.nn.sparse_softmax_cross_entropy_with_logits(labels=y, logits=logits))
train_op = tf.compat.v1.train.AdamOptimizer(1e-3).minimize(loss)
preds = tf.argmax(logits, axis=1)
acc = tf.reduce_mean(tf.cast(tf.equal(preds, tf.cast(y, tf.int64)), tf.float32))
init = tf.compat.v1.global_variables_initializer()
return ("graph":g, "X":X, "y":y, "loss":loss, "train_op":train_op, "acc":acc, "init":init)

def train_graph_v1(graph_dict, Xtr, ytr, Xte, yte, epochs=8, batch=128, name="RNN"):
    g = graph_dict["graph"]
    with tf.compat.v1.Session(graph=g) as sess:
        sess.run(graph_dict["init"])
        n = Xtr.shape[0]
        for ep in range(1, epochs+1):
            idx = np.random.permutation(n)
            Xp, yp = Xtr[idx], ytr[idx]
            losses=[]
            for i in range(0, n, batch):
                xb = Xp[i:i+batch]; yb = yp[i:i+batch]
                _, l = sess.run([graph_dict["train_op"], graph_dict["loss"]],
                               feed_dict={graph_dict["X"]:xb, graph_dict["y"]:yb})
            losses.append(l)
            acc_val = sess.run(graph_dict["acc"], feed_dict={graph_dict["X"]:Xte, graph_dict["y"]:yte})
            print(f"Epoch {ep:02d} | loss={np.mean(losses):.4f} | acc_val={acc_val:.3f}")
        return float(acc_val)

print_bar("TRAIN CNN / RNN / LSTM (graph mode)")
acc_cnn_val = train_cnn_graph(epochs=8, batch=128)
rnn_graph = build_stacked_rnn_graph(feast_dim, hidden_hidden_size, layers=num_layers, n_classes=num_classes)
lstm_graph = build_stacked_lstm_graph(feast_dim, hidden_hidden_size, layers=num_layers, n_classes=num_classes)
acc_rnn_val = train_graph_v1(rnn_graph, X_train_t, y_train_id, X_test_t, y_test_id, epochs=8, batch=128, name="RNN")
acc_lstm_val = train_graph_v1(lstm_graph, X_train_t, y_train_id, X_test_t, y_test_id, epochs=8, batch=128, name="LSTM")

# 5) SO SÁNH KẾT QUẢ (e)
# -----
best_ml_name = max(ml_results, key=lambda k: ml_results[k]["acc"])
best_ml = ml_results[best_ml_name]
print_bar("ML SUMMARY")
for k, v in ml_results.items():

```

Hình 21

```

Wo = tf.Variable(tf.random_normal([hidden, n_classes], stddev=0.1), name="Wo")
bo = tf.Variable(tf.zeros([n_classes]), name="bo")
logits = tf.matmul(last_h, Wo) + bo

loss = tf.reduce_mean(tf.nn.sparse_softmax_cross_entropy_with_logits(labels=y, logits=logits))
train_op = tf.compat.v1.train.AdamOptimizer(1e-3).minimize(loss)
preds = tf.argmax(logits, axis=1)
acc = tf.reduce_mean(tf.cast(tf.equal(preds, tf.cast(y, tf.int64)), tf.float32))
init = tf.compat.v1.global_variables_initializer()
return ("graph":g, "X":X, "y":y, "loss":loss, "train_op":train_op, "acc":acc, "init":init)

def train_graph_v1(graph_dict, Xtr, ytr, Xte, yte, epochs=8, batch=128, name="RNN"):
    g = graph_dict["graph"]
    with tf.compat.v1.Session(graph=g) as sess:
        sess.run(graph_dict["init"])
        n = Xtr.shape[0]
        for ep in range(1, epochs+1):
            idx = np.random.permutation(n)
            Xp, yp = Xtr[idx], ytr[idx]
            losses=[]
            for i in range(0, n, batch):
                xb = Xp[i:i+batch]; yb = yp[i:i+batch]
                _, l = sess.run([graph_dict["train_op"], graph_dict["loss"]],
                               feed_dict={graph_dict["X"]:xb, graph_dict["y"]:yb})
            losses.append(l)
            acc_val = sess.run(graph_dict["acc"], feed_dict={graph_dict["X"]:Xte, graph_dict["y"]:yte})
            print(f"Epoch {ep:02d} | loss={np.mean(losses):.4f} | acc_val={acc_val:.3f}")
        return float(acc_val)

print_bar("TRAIN CNN / RNN / LSTM (graph mode)")
acc_cnn_val = train_cnn_graph(epochs=8, batch=128)
rnn_graph = build_stacked_rnn_graph(feast_dim, hidden_hidden_size, layers=num_layers, n_classes=num_classes)
lstm_graph = build_stacked_lstm_graph(feast_dim, hidden_hidden_size, layers=num_layers, n_classes=num_classes)
acc_rnn_val = train_graph_v1(rnn_graph, X_train_t, y_train_id, X_test_t, y_test_id, epochs=8, batch=128, name="RNN")
acc_lstm_val = train_graph_v1(lstm_graph, X_train_t, y_train_id, X_test_t, y_test_id, epochs=8, batch=128, name="LSTM")

# 5) SO SÁNH KẾT QUẢ (e)
# -----
best_ml_name = max(ml_results, key=lambda k: ml_results[k]["acc"])
best_ml = ml_results[best_ml_name]
print_bar("ML SUMMARY")
for k, v in ml_results.items():

```

Hình 22

➔ So sánh cấu trúc CNN, RNN, LSTM

- Mạng no-ron tích chập (CNN – Convolutional Neural Network)

CNN mô phỏng cách hệ thống thị giác của con người hoạt động, tập trung vào việc trích xuất các đặc trưng từ dữ liệu đầu vào.

- Lớp tích chập (Convolutional Layer): Đây là thành phần cốt lõi. Lớp này sử dụng các bộ lọc (filters hoặc kernels) trượt qua hình ảnh đầu vào để thực hiện các phép tích chập. Mỗi bộ lọc được thiết kế để phát hiện một đặc trưng cụ thể, chẳng hạn như cạnh, góc, màu sắc. Quá trình này tạo ra một bản đồ đặc trưng (feature map)
 - Hàm kích hoạt (Activation Function – thường là ReLu): Sau lớp tích chập, hàm ReLu được áp dụng để đưa yếu tố phi tuyến vào mô hình, giúp mạng học được các mối quan hệ phức tạp hơn
 - Lớp gộp (Pooling Layer) Lớp này làm giảm kích thước không gian (chiều rộng và chiều cao của bản đồ đặc trưng, giúp giảm số lượng tham số và tính toán trong mạng. Quá trình này cũng phép biến đổi trở nên “bất biến” với những thay đổi nhỏ về vị trí đặc trưng. Phép gộp phổ biến nhất là Max Pooling.
 - Lớp kết nối đầy đủ (Fully Connected layer): sau khi các đặc trưng được trích xuất và giảm chiều qua nhiều lớp tích chập và gộp, chúng được làm phẳng (flatten) thành một vector và đưa vào một hoặc nhiều lớp kết nối đầy đủ. Lớp này có nhiệm vụ kết hợp các đặc trưng lại để thực hiện việc phân loại cuối cùng.
- Mạng no-ron hồi quy (RNN – Recurrent Neural Network)

RNN được thiết kế để nhận dạng các mẫu trong dữ liệu tuần tự. Điểm đặc biệt của nó là sự tồn tại của một vòng lặp.

- Vòng lặp (Loop): Tại mỗi bước của thời gian t , mạng không chỉ nhận vào đầu vào x mà còn nhận cả đầu ra (trạng thái ẩn – hidden state) h từ bước thời gian trước đó.
- Trạng thái ẩn (hidden state): Hoạt động như 1 bộ nhớ của mạng, lưu trữ thông tin về những gì đã xảy ra trong các bước trước đó. Nó được tính toán dựa trên đầu vào hiện tại x và trạng thái ẩn trước đó.

$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- Chia sẻ trọng số (Weight Sharing): Cùng một bộ trọng số (W, U) và hàm kích hoạt được sử dụng ở mọi bước thời gian. Điều này giúp giảm đáng kể tổng số tham số mà mô hình cần học.

Vấn đề của RNN: DO thông tin được truyền qua nhiều bước, RNN gặp phải vấn đề suy giảm / bùng nổ gradient (vanishing/ exploding gradient). Điều này khiến mạng khó học được các phụ thuộc xa trong một chuỗi dài (ví dụ: Mối liên hệ giữa từ đầu tiên và từ cuối cùng của một đoạn văn dài)

- Bộ nhớ dài ngắn (Long Short-Term Memory)

LSTM là một biến thể đặc biệt của RNN, được tạo ra để giải quyết vấn đề về bộ nhớ dài hạn của cấu trúc RNN thông thường.

Cấu trúc của LSTM phức tạp hơn RNN với thành phần cốt lõi là ô nhớ (cell). Mỗi ô nhớ có một trạng thái ô nhớ (cell state) và ba cổng (gates) để điều khiển luồng thông tin.

- Trạng thái ô nhớ (Ct): Đây được coi là “bộ nhớ dài hạn” của mạng. Nó chạy dọc theo toàn bộ chuỗi với ít thay đổi tuyến tính. Thông tin có thể dễ dàng được thêm vào hoặc loại bỏ khỏi cell state thông qua các cổng.
- Các cổng (gates): Các cổng này là các mạng no-ron nhỏ (thường dùng hàm sigmoid) quyết định thông tin nào nên được đi qua.
 - Cổng quên (forget gate): Quyết định thông tin nào sẽ bị loại bỏ khỏi cell state của bước trước. Nó nhìn vào h và x để đưa ra quyết định
 - Cổng đầu vào (Input gate): Quyết định thông tin mới nào sẽ được lưu trữ trong cell state. Nó bao gồm hai phần: một lớp sigmoid quyết định giá trị nào cần cập nhật và một lớp tanh tạo ra một vector chứa các giá trị ứng viên mới.
 - Cổng đầu ra (output gate): Quyết định phần nào của cell state sẽ được đưa làm đầu ra (trạng thái ẩn h). trạng thái cell state sẽ đi qua một hàm tanh (để đưa giá trị về khoảng -1 đến 1) và sau đó được nhân với đầu ra của cổng sigmoid.
- ⇒ Batch size là số mẫu đưa vào mạng trong một lần cập nhật tham số.
 - Batch size nhỏ (16, 32): gradient nhiều hơn → có tác dụng regularization tự nhiên nhưng chạy chậm hơn
 - Batch size lớn (128, 256): tính toán ổn định, tận dụng GPU tốt, nhưng dễ bị kẹt ở nghiệm kém (generalization yếu)
- ⇒ Learning rate là tốc độ học, bước nhảy khi cập nhật trọng số.
 - LR quá lớn → loss dao động, có thể diverge
 - LR quá nhỏ → học chậm, kẹt ở local minima.
 - Thường dùng giá trị vừa phải ($1e-3$ với Adam, $1e-2$ với SGD)
- ⇒ Epoch là số lần duyệt hết toàn bộ tập train.
 - Ít epoch → underfitting
 - Quá nhiều epoch → overfitting
 - Thường kết hợp với early stopping
- ⇒ Ảnh hưởng khi thay đổi LR/ Batch/ Epoch

Tham số	Tăng lên	Giảm xuống
Batch size	Tính toán nhanh hơn / epoch, gradient ổn định hơn nhưng dễ mất tính đa dạng → overfit	Gradient nhiều, generalization tốt hơn nhưng tính toán lâu hơn

Learning rate	Hội tụ nhanh hơn lúc đầu vào, dễ dao động	Hội tụ chậm
Epoch	Train acc tăng, dễ overfit	Underigt, chưa học đủ

⇒ Data vào CNN, RNN, LSTM

- Dữ liệu gốc sau tiền xử lý $X_{train_t}.shape = [B, F]$ với B là batch size và F là số đặc trưng (feature_dim)
- Đưa vào CNN

```
Xc = tf.compat.v1.placeholder(tf.float32, shape=[None, feat_dim], name="X")
x1 = tf.expand_dims(Xc, -1) # [B, T=F, C=1]
```

- o Ban đầu $Xc = [B, F]$
- o Sau `expand_dims(Xc, -1)` → thêm trục kênh (channel) $[B, T = F, C=1]$
- o T là độ dài chuỗi = số đặc trưng F
- o C là số kênh (ban đầu chỉ 1)

⇒ CNN coi các đột đặc trưng như một chuỗi → quét kernel dọc theo trục T

- Đưa vào RNN

```
X = tf.compat.v1.placeholder(tf.float32, shape=[None, feat_dim], name="X")
x_seq = tf.expand_dims(X, -1) # [B, T=F, D=1]
```

- o Ban đầu $Xc = [B, F]$
- o Sau `expand_dims` → $[B, T = F, D = 1]$
- o T là số bước thời gian = số đặc trưng
- o D = 1 (mỗi bước chỉ có 1 giá trị)

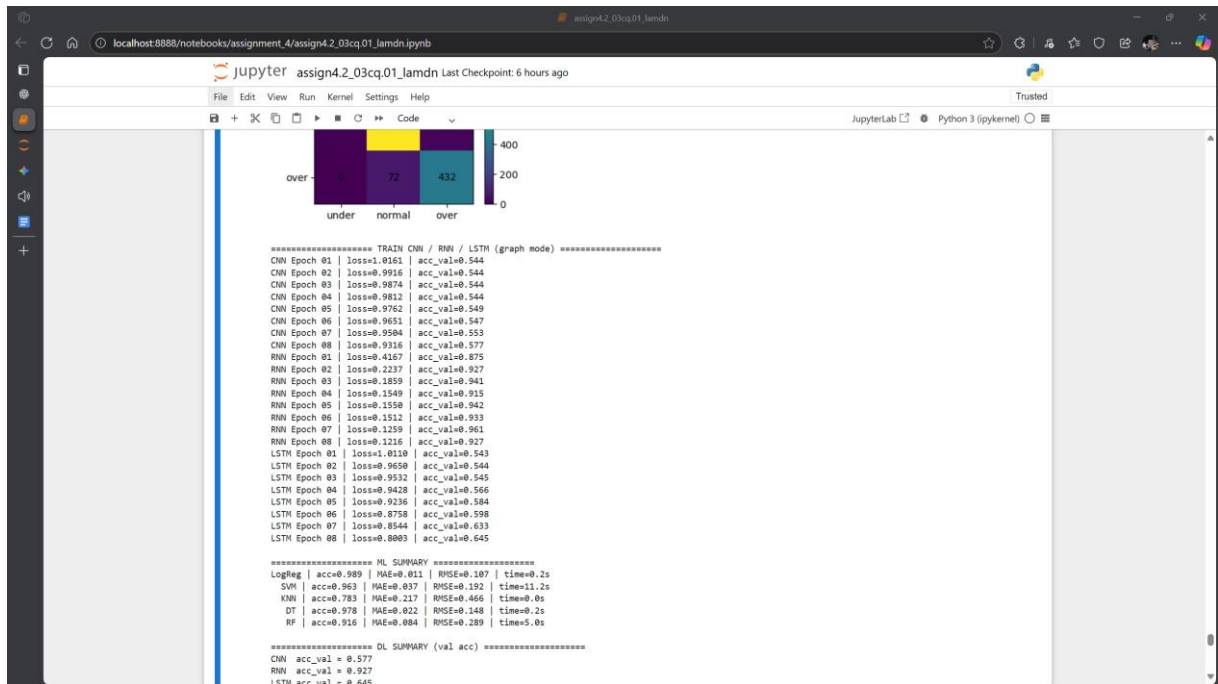
⇒ RNN sẽ duyệt theo chiều F: ở mỗi bước t, lấy 1 đặc trưng và cập nhật trạng thái ẩn

- Đưa vào LSTM

```
X = tf.compat.v1.placeholder(tf.float32, shape=[None, feat_dim], name="X")
x_seq = tf.expand_dims(X, -1) # [B, T=F, D=1]
```

- o Giống RNN $[B, T = F, D = 1]$
- o LSTM duyệt qua từng bước t nhưng khác RNN ở chỗ nó lưu thêm cell state và điều khiển qua cổng i/f/o

⇒ Kết quả



Hình 23

e) Compare and evaluate models given with metrics accuracy, MAE, MSE, RMSE

```

print(f"(name) Epoch (ep:020) | loss={np.mean(losses):.4f} | acc_val={acc_val:.3f}")
return float(acc_val)

print_bar("TRAIN CNN / RNN / LSTM (graph mode)")
acc_cnn_val = train_cnn_graph(epochs=8, batch=128)
rnn_graph = build_stacked_rnn_graph(feet_dim, hidden_hidden_size, layers=num_layers, n_classes=num_classes)
lstm_graph = build_stacked_lstm_graph(feet_dim, hidden_hidden_size, layers=num_layers, n_classes=num_classes)
acc_rnn_val = train_graph_vl(rnn_graph, X_train_t, y_train_id, X_test_t, y_test_id, epochs=8, batch=128, name="RNN")
acc_lstm_val = train_graph_vl(lstm_graph, X_train_t, y_train_id, X_test_t, y_test_id, epochs=8, batch=128, name="LSTM")

# 5) SO SÁNH KẾT QUẢ (e)
# -----
best_ml_name = max(ml_results, key=lambda k: ml_results[k]["acc"])
best_ml = ml_results[best_ml_name]
print_bar("ML SUMMARY")
for k, v in ml_results.items():
    print(f"(k:{k}) | acc={v['acc']:.3f} | MAE={v['mae']:.3f} | RMSE={v['rmse']:.3f} | time={v['time_s']:.1f}s")

print_bar("DL SUMMARY (val acc)")
print(f"CNN acc_val = {acc_cnn_val:.3f}")
print(f"RNN acc_val = {acc_rnn_val:.3f}")
print(f"LSTM acc_val = {acc_lstm_val:.3f}")

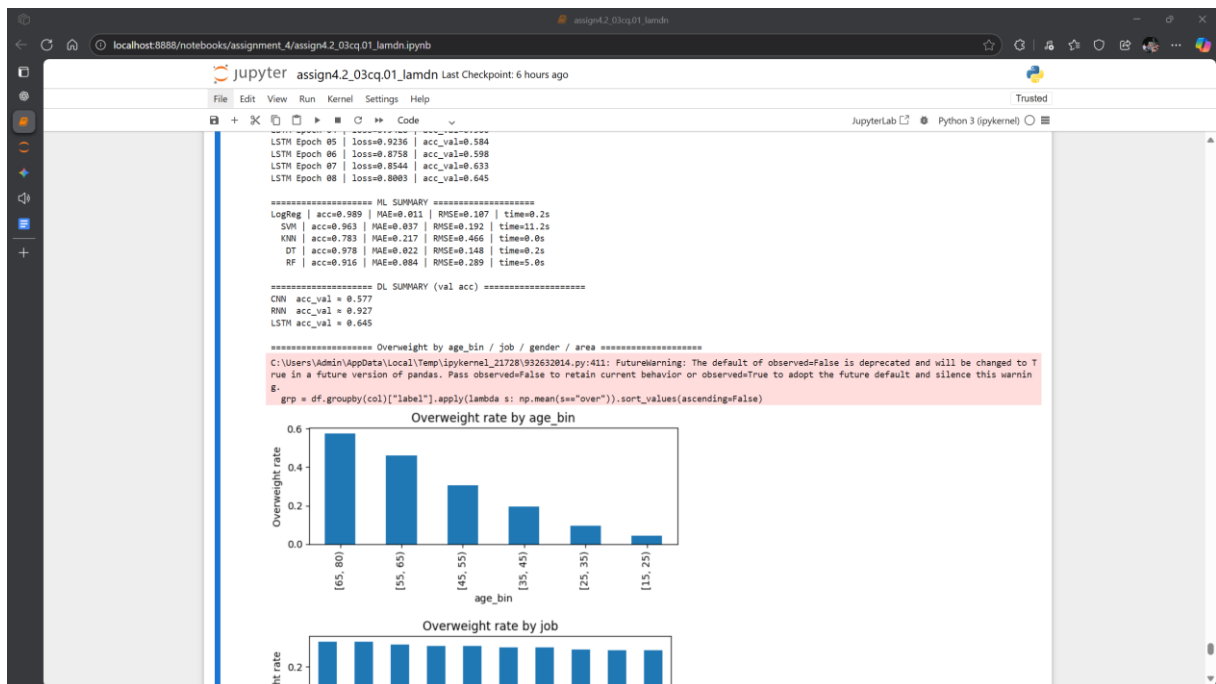
# 6) VIZ: NHÌN THẤU CẤU (f)
# -----
df["age_bin"] = pd.cut(df["age"], bins=[15,25,35,45,55,65,80], right=False)
def bar_counts(col):
    grp = df.groupby(col)["label"].apply(lambda s: np.mean(s=="over")).sort_values(ascending=False)
    plt.figure(figsize=(6,3))
    grp.plot(kind="bar"); plt.ylabel("Overweight rate")
    plt.title(f"Overweight rate by {col}"); plt.tight_layout(); plt.show()
    return grp

print_bar("Overweight by age_bin / job / gender / area")
ov_age = bar_counts("age_bin")
ov_job = bar_counts("job")
ov_gender = bar_counts("gender")
ov_area = bar_counts("area")

# 7) CHỌN MÔ HÌNH TỐT NHẤT & ĐỀ XUẤT (g)

```

Hình 24



Hình 25

⇒ LogReg độ chính xác cao nhất với thời gian chạy chậm nhất

f) Visualize the results:

- Which age has more overweight persons?
- Which job has more overweight persons?
- Which gender has more overweight persons?
- Which area has more overweight persons?

```
jupyter assign4.2_03cq.01_lamdn Last Checkpoint: 7 hours ago

File Edit View Run Kernel Settings Help

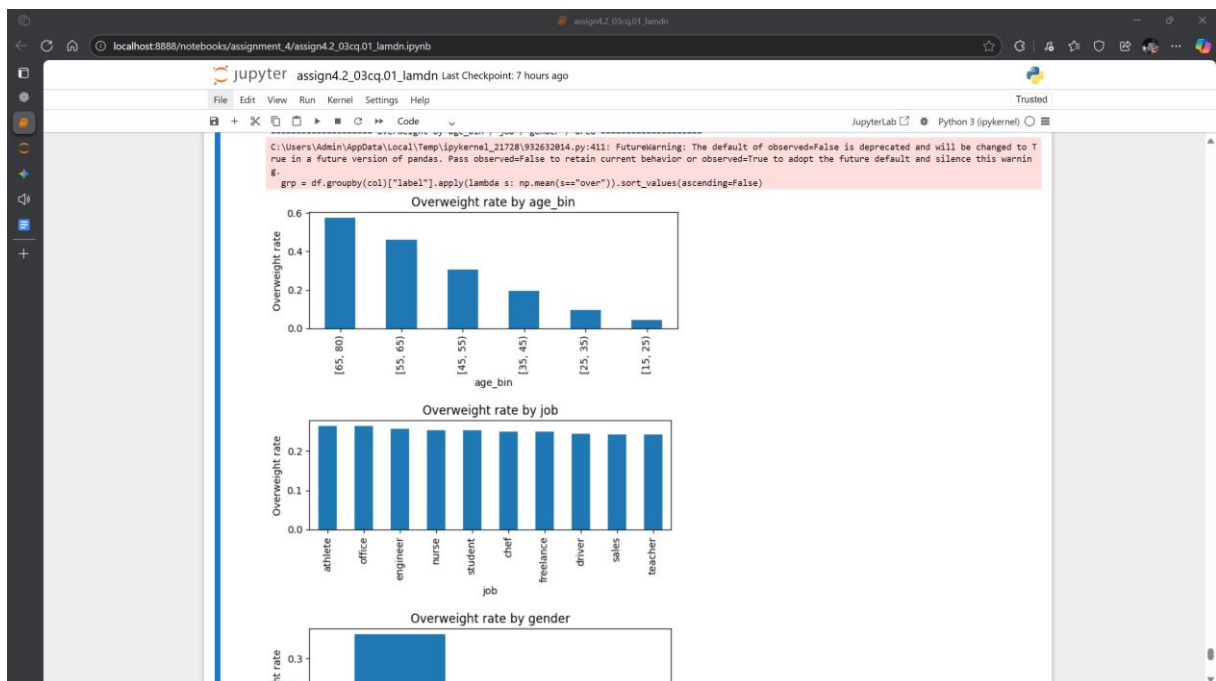
# 6) VIZ: NHON THUA CAN (F)
# -----
df["age_bin"] = pd.cut(df["age"], bins=[15,25,35,45,55,65,80], right=False)
def bar_counts(col):
    grp = df.groupby(col).apply(lambda s: np.mean(s=="over")).sort_values(ascending=False)
    plt.figure(figsize=(6,3))
    grp.plot(kind="bar"); plt.ylabel("Overweight rate")
    plt.title(f"Overweight rate by {col}"); plt.tight_layout(); plt.show()
    return grp

print_bar("Overweight by age_bin / job / gender / area")
ov_age = bar_counts("age_bin")
ov_job = bar_counts("job")
ov_gender = bar_counts("gender")
ov_area = bar_counts("area")

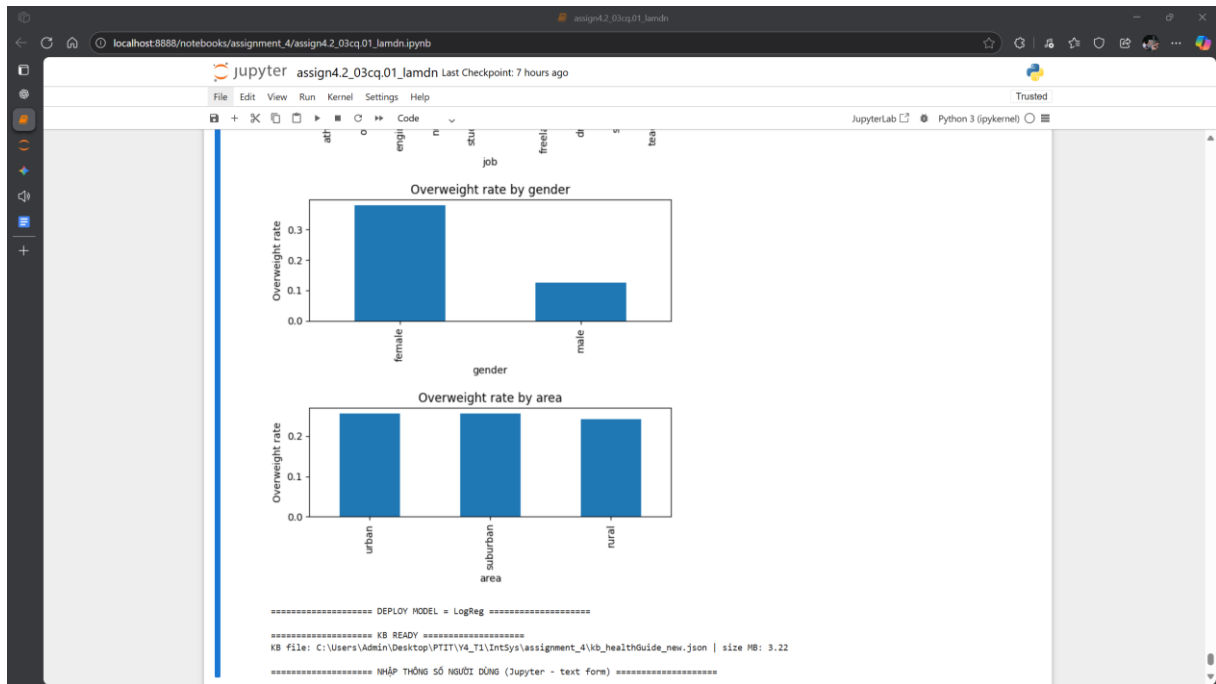
# 7) CHON MO HINH TOT NHAT & DEPLOY (g)
# -----
overall_best = best_ml # dung mo hinh ML tot nhut de deploy
deploy_pipe = overall_best("pipe")
print_bar(f"DEPLOY MODEL = {best_ml_name}")

def deploy_predict(user_dict):
    """
    user_dict keys: job, gender, area, diet, activity, age, sleep_h, steps, height_cm,
    weight_kg, sugar_g, fat_g, protein_g, fiber_g, waist_cm,
    bp_sys, bp_dia, cholesterol, smoker, alcohol_units
    Tra ve: label, probs
    """
    row = []
    for c in X.columns:
        if c in user_dict:
            row.append(user_dict[c])
        else:
            row.append(df[c].median() if c in num_cols else df[c].mode()[0])
    X1 = pd.DataFrame(row, columns=X.columns)
    pred = deploy_pipe.predict(X1)[0]
    if isinstance(pred, float):
        return pred
```

Hình 26



Hình 27



Hình 28

- ➔ Age group bị overweight nhiều nhất là 65 đến 80
- ➔ Job bị overweight nhiều nhất là althete
- ➔ Gender bị overweight nhiều nhất là Female
- ➔ Area bị overweight nhiều nhất là urban

g) Deploy the best model so that user enters features and output is underweight, overweight or normal.

```

return grp

print_bar("Overweight by age_bin / job / gender / area")
ov_age = bar_counts("age_bin")
ov_job = bar_counts("job")
ov_gender = bar_counts("gender")
ov_area = bar_counts("area")

# -----
# 7) CHỌN MÔ HÌNH TỐT NHẤT & DEPLOY (g)
# -----
overall_best = best_ml # dùng mô hình ML tốt nhất để deploy
deploy_pipe = overall_best["pipe"]
print_bar(f"DEPLOY MODEL = (best_ml_name)")

def deploy_predict(user_dict):
    """
    user_dict keys: job, gender, area, diet, activity, age, sleep_h, steps, height_cm,
    weight_kg, sugar_g, fat_g, protein_g, fiber_g, waist_cm,
    bp_sys, bp_dia, cholesterol, smoker, alcohol_units
    Trả về: label, probs
    """
    row = []
    for c in X.columns:
        if c in user_dict:
            row.append(user_dict[c])
        else:
            row.append(df[c].median() if c in num_cols else df[c].mode()[0])
    X1 = pd.DataFrame(row, columns=X.columns)
    pred = deploy_pipe.predict(X1)[0]
    if hasattr(deploy_pipe, "predict_proba"):
        pr = deploy_pipe.predict_proba(X1)[0]
        probs = (lbl: float(pr[i]) for i, lbl in enumerate(deploy_pipe.classes_))
    else:
        probs = (lbl: 1.0/len(deploy_pipe.classes_) for lbl in deploy_pipe.classes_)
    return pred, probs

# -----
# 8) SINH BKG KNOWLEDGE BASE (>3MB) (h)
# -----
def build_bkg_kb(min_size_mb=3.2, seed=42, verbose=True):
    rng = random.Random(seed)
    kb = {}

```

Hình 29

```

===== DEPLOY MODEL = LogReg =====

===== KB READY =====
KB file: C:\Users\Admin\Desktop\PTIT\Y4_T1\IntSys\assignment_4\kb_healthGuide_new.json | size MB: 3.22

===== NHẬP THÔNG SỐ NGƯỜI DÙNG (Jupyter - text form) =====
job [nurse] options=['athlete', 'chef', 'driver', 'engineer', 'freelance', 'nurse', 'office', 'sales', 'student', 'teacher']:
[ ]:

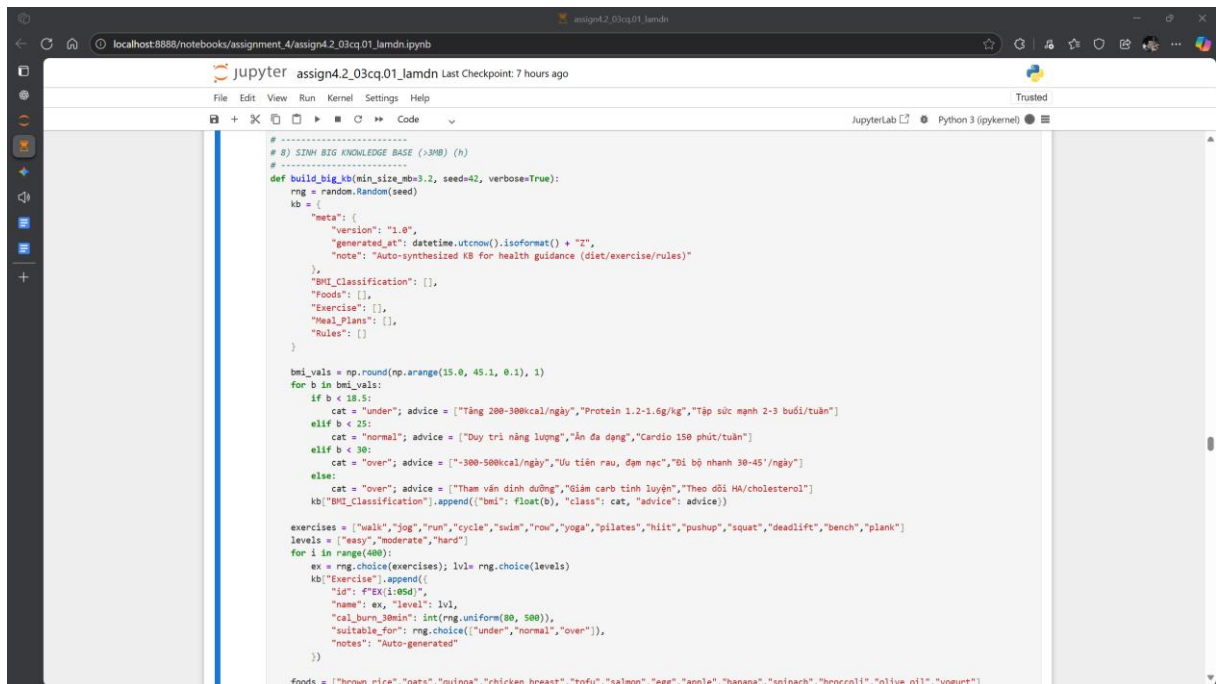
```

Hình 30

➔ Giải thích code

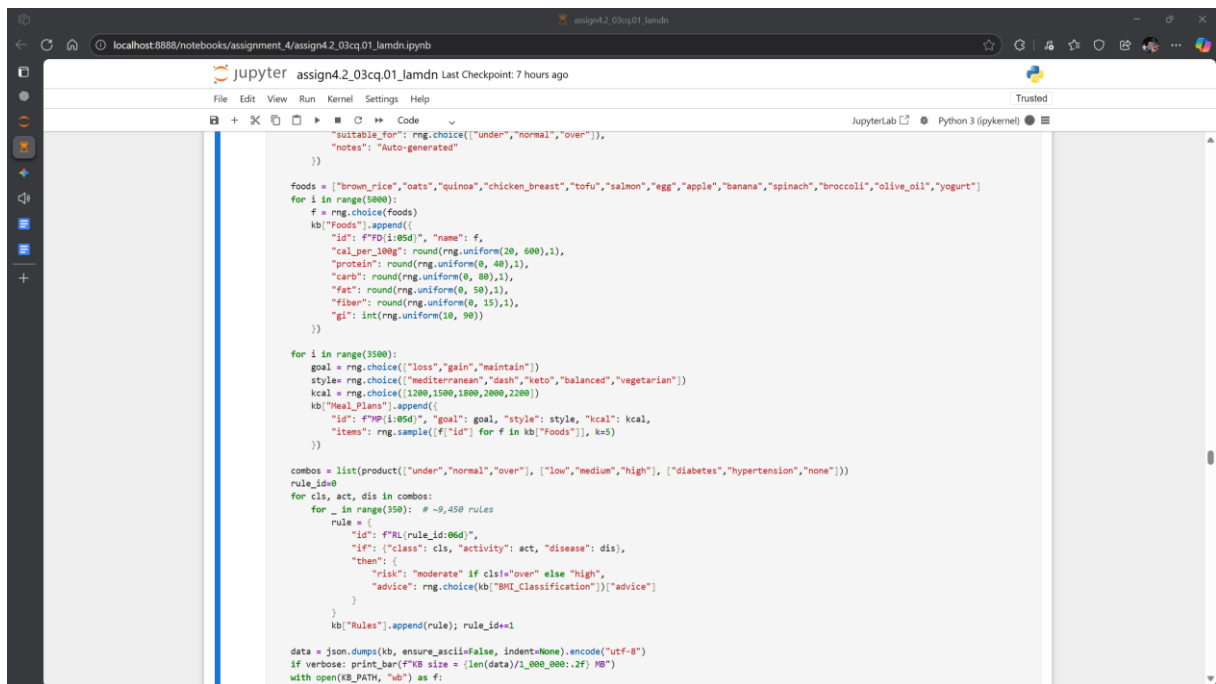
- Chọn mô hình có acc cao nhất ➔ best_ml
- Lấy luôn cả pipeline của mô hình tốt nhất, bao gồm (pre xử lý và clf mô inh)
- Hàm deploy_predict
 - User_dicts: chứa thông tin người dung nhập
 - Output: label dự đoán và xác suất từng lớp

h) Build a BIG knowledge base more 3MB and store and give a guider for health based on this knowledge base.



```
# -----  
# 8) SINH BIG KNOWLEDGE BASE (>3MB) (h)  
# -----  
def build_big_kb(min_size_mb=3.2, seed=42, verbose=True):  
    rng = random.Random(seed)  
    kb = {  
        "Meta": {  
            "version": "1.0",  
            "generated_at": datetime.utcnow().isoformat() + "Z",  
            "note": "Auto-synthesized KB for health guidance (diet/exercise/rules)"  
        },  
        "BMI_Classification": [],  
        "Foods": [],  
        "Exercise": [],  
        "Meal_Plans": [],  
        "Rules": []  
    }  
  
    bmi_vals = np.round(np.arange(15.0, 45.1, 0.1), 1)  
    for b in bmi_vals:  
        if b < 18.5:  
            cat = "under"; advice = ["Tăng 200-300kcal/ngày", "Protein 1.2-1.6g/kg", "Tập sức mạnh 2-3 buổi/tuần"]  
        elif b < 25:  
            cat = "normal"; advice = ["Duy trì năng lượng", "Ăn đa dạng", "Cardio 150 phút/tuần"]  
        elif b < 30:  
            cat = "over"; advice = ["-300-500kcal/ngày", "Uu tiên rau, đạm nạc", "Đi bộ nhanh 30-45'/ngày"]  
        else:  
            cat = "over"; advice = ["Tham vấn dinh dưỡng", "Giảm carb tinh luyện", "Theo dõi HA/cholesterol"]  
        kb["BMI_Classification"].append({"bmi": float(b), "class": cat, "advice": advice})  
  
    exercises = ["walk", "jog", "run", "cycle", "swim", "row", "yoga", "pilates", "hitt", "pushup", "squat", "deadlift", "bench", "plank"]  
    levels = ["easy", "moderate", "hard"]  
    for i in range(400):  
        ex = rng.choice(exercises); lvl = rng.choice(levels)  
        kb["Exercise"].append({  
            "id": f"EX{i:05d}",  
            "name": ex, "level": lvl,  
            "cal_burn_30min": int(rng.uniform(80, 500)),  
            "suitable_for": rng.choice(["under", "normal", "over"]),  
            "notes": "Auto-generated"  
        })  
  
    foods = ["brown_rice", "oats", "quinoa", "chicken_breast", "tofu", "salmon", "egg", "apple", "banana", "spinach", "broccoli", "olive_oil", "yogurt"]
```

Hình 31



```
        "suitable_for": rng.choice(["under", "normal", "over"]),  
        "notes": "Auto-generated"  
    })  
  
    foods = ["brown_rice", "oats", "quinoa", "chicken_breast", "tofu", "salmon", "egg", "apple", "banana", "spinach", "broccoli", "olive_oil", "yogurt"]  
    for i in range(5000):  
        f = rng.choice(foods)  
        kb["Foods"].append({  
            "id": f"FO{i:05d}", "name": f,  
            "cal_per_100g": round(rng.uniform(20, 600), 1),  
            "protein": round(rng.uniform(0, 40), 1),  
            "carb": round(rng.uniform(0, 80), 1),  
            "fat": round(rng.uniform(0, 50), 1),  
            "fiber": round(rng.uniform(0, 15), 1),  
            "gi": int(rng.uniform(10, 90))  
        })  
  
    for i in range(3500):  
        goal = rng.choice(["loss", "gain", "maintain"])  
        style = rng.choice(["mediterranean", "dash", "keto", "balanced", "vegetarian"])  
        kcal = rng.choice([1200, 1500, 1800, 2000, 2200])  
        kb["Meal_Plans"].append({  
            "id": f"MP{i:05d}", "goal": goal, "style": style, "kcal": kcal,  
            "items": rng.sample([f"{id}" for f in kb["Foods"]], k=5)  
        })  
  
    combos = list(product(["under", "normal", "over"], ["low", "medium", "high"], ["diabetes", "hypertension", "none"]))  
    rule_id=0  
    for _ in range(350): # ~9,450 rules  
        rule = {  
            "id": f"RL{rule_id:06d}",  
            "if": {"class": cls, "activity": act, "disease": dis},  
            "then": {  
                "risk": "moderate" if cls=="over" else "high",  
                "advice": rng.choice(kb["BMI_Classification"])[0]["advice"]  
            }  
        }  
        kb["Rules"].append(rule); rule_id+=1  
  
    data = json.dumps(kb, ensure_ascii=False, indent=None).encode("utf-8")  
    if verbose: print_bar(f"KB size = {len(data)/1_000_000:.2f} MB")  
    with open(KB_PATH, "wb") as f:
```

Hình 32

```
for i in range(3500):
    goal = rng.choice(["loss", "gain", "maintain"])
    style = rng.choice(["mediterranean", "dash", "keto", "balanced", "vegetarian"])
    kcal = rng.choice([1200, 1500, 1800, 2000, 2200])
    kb["Meal_Plans"].append({
        "id": f"MP{i:05d}", "goal": goal, "style": style, "kcal": kcal,
        "items": rng.sample([f"{id}" for f in kb["Foods"]], k=5)
    })

combos = list(product(["under", "normal", "over"], ["low", "medium", "high"], ["diabetes", "hypertension", "none"]))
rule_id=0
for cls, act, dis in combos:
    for _ in range(350): # ~9,450 rules
        rule = {
            "id": f"R{rule_id:06d}",
            "if": {"class": cls, "activity": act, "disease": dis},
            "then": {
                "risk": "moderate" if cls=="over" else "high",
                "advice": rng.choice(kb["BMI_Classification"])["advice"]
            }
        }
        kb["Rules"].append(rule); rule_id+=1

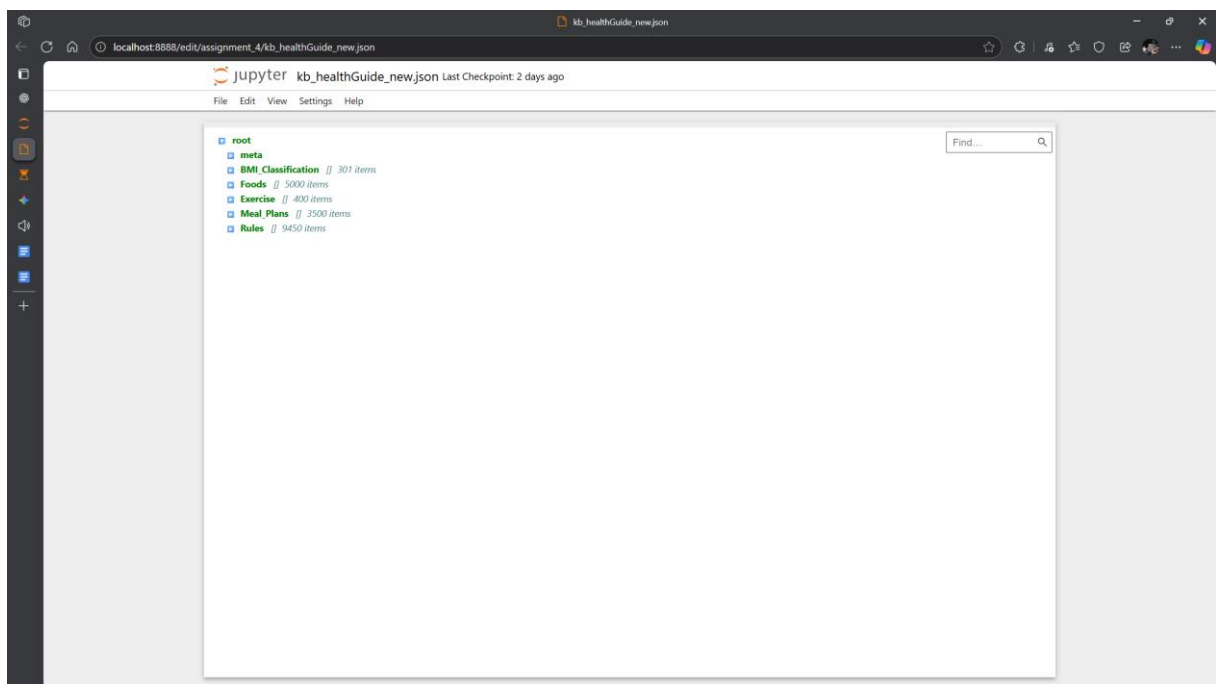
data = json.dumps(kb, ensure_ascii=False, indent=None).encode("utf-8")
if verbose: print_bar(f"KB size = {len(data)/1_000_000:.2f} MB")
with open(KB_PATH, "wb") as f:
    f.write(data)
return kb

if (not os.path.exists(KB_PATH)) or (os.path.getsize(KB_PATH) < 3_000_000):
    kb = build_big_kb(min_size_mb=3.2)
else:
    with open(KB_PATH, "rb") as f:
        kb = json.loads(f.read().decode("utf-8", errors="ignore"))

print_bar("KB READY")
print(f"KB file: {KB_PATH, ' '} size MB: {round(os.path.getsize(KB_PATH)/1_000_000, 2)}")

# -----
# 8.1 HÀM kb_chat - chọn gợi ý từ KB (FX: bổ sung)
def kb_chat(context, predicted_label):
    """
```

Hình 33



Hình 34

➔ Giải thích code

- Hàm build_big_kb

- Random seed ➔ dữ liệu sinh ra ổn định giữa các lần chạy
 - Kb là dict “khung xương” gồm 5 mảng dữ liệu + thông tin
- Sinh bảng phân loại BMI ➔ lời khuyên
- Sinh 361 mức BMI từ 15.0 ➔ 45.0
 - Gán class theo ngưỡng chuẩn + gợi ý nhanh theo nhóm

- Mục đích: khi dự đoán xong, có thể tra BMI gần nhất để rút gợi ý phù hợp
- Danh mục bài tập:
 - Tạo 400 bản ghi, mỗi bản ghi có id, name, level, calo tiêu hao trong 30p, phù hợp với (ngẫu nhiên trong under, normal, over)
- Danh mục thực phẩm:
 - Tạo 5000 mục thực phẩm với macro giả lập (cal_per_100g, protein, carb, fat, fiber, gi)
- Tạo thực đơn
 - 3500 thực đơn
- Luật
 - Có 27 tổ hợp điều kiện (3BMI class x 3 activity x 3 disease)
 - Mỗi tổ hợp sinh 350 luật $\rightarrow 27 \times 350 = 9450$ luật
 - Risk nếu class là over $\rightarrow$ high, ngược lại: moderate
 - Advice
- Lưu vào file Json

i) **Build guide with chat based on your knowledge base and prediction result**



The screenshot shows a JupyterLab interface with a file named 'assign4.2_03cq.01_lamdn.ipynb'. The code in the notebook is as follows:

```

# 1) Theo BMI gần nhất (nếu có)
nearest_bmi = None
if bmi is not None and isinstance(kb, dict) and "BMI_Classification" in kb and kb["BMI_Classification"]:
    arr = kb["BMI_Classification"]
    idx = int(np.argmax([abs(bmi - e["bmi"]) for e in arr]))
    nearest_bmi = float(arr[idx]["bmi"])
    bmi_adv = list(arr[idx].get("advice", []))
    out_adv.extend(bmi_adv)

# 2) Theo Rule khớp (class = predicted_label, activity, disease)
rules = kb.get("Rules", []) if isinstance(kb, dict) else []
if rules:
    count = 0

```

Hình 35

The screenshot shows a JupyterLab environment with a Python 3 (ipykernel) session. The code in the notebook is as follows:

```
arr = np.unique(arr)
idx = int(np.argmax([abs(bmi - e["bmi"]) for e in arr]))
nearest_bmi = float(arr[idx]["bmi"])
bmi_adv = list(arr[idx].get("advice", []))
out_adv.extend(bmi_adv)

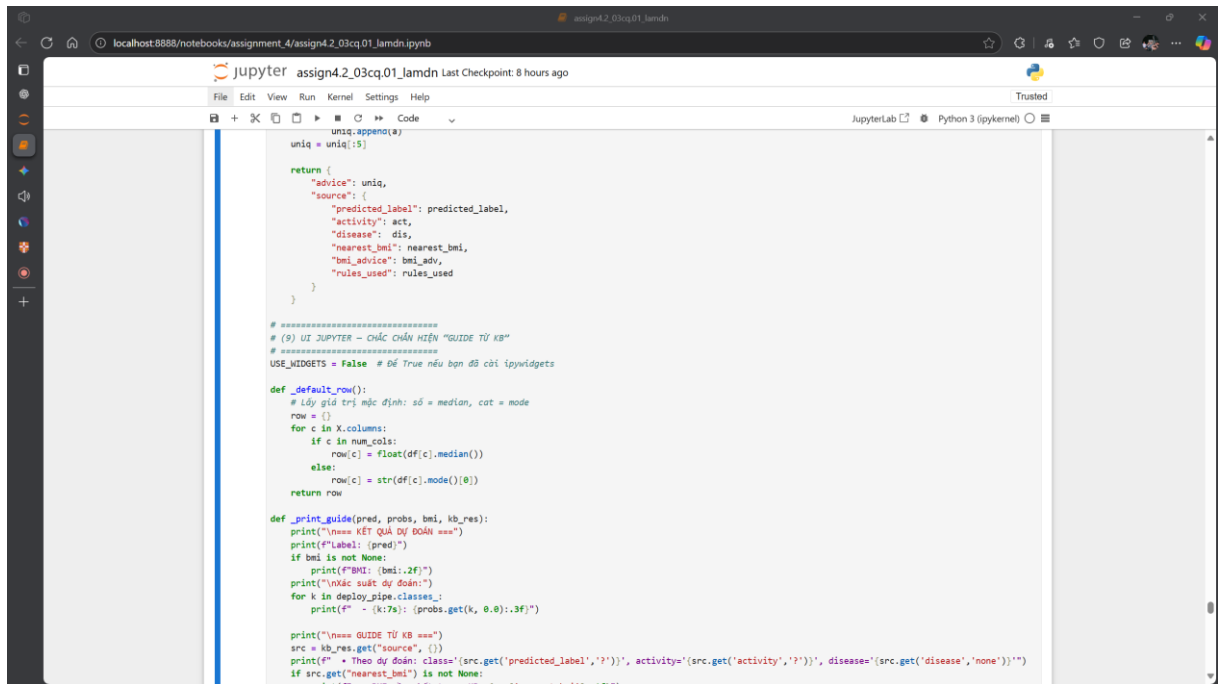
# 2) Theo Rule book (class = predicted_label, activity, disease)
rules = kb.get("Rules", []) if isinstance(kb, dict) else []
if rules:
    count = 0
    for r in rules:
        cond = r.get("if", {})
        if cond.get("class")==predicted_label and cond.get("activity")==act and cond.get("disease")==dis:
            adv_r = r.get("then", {}).get("advice", [])
            if adv_r:
                out_adv.extend(adv_r)
            rules_used.append(("id": r.get("id", ""), "advice": list(adv_r)))
            count += 1
            if count >= 3: # Lấy tối đa 3 rule để đa dạng nhưng gọn
                break

# 3) Fallback nếu vẫn rỗng
if not out_adv:
    out_adv = ["Uống đủ nước (≥ 1.5-2L/ngày)",
               "Ngủ 7-8 giờ/đêm, giờ lịch ngủ đều",
               "Ăn rau xanh & trái cây",
               "Giảm đồ ngọt & nước có ga",
               "Đi bộ nhanh 30 phút/ngày"]

# Loại trùng, Lấy top 5
uniq = []
for a in out_adv:
    if a not in uniq:
        uniq.append(a)
uniq = uniq[:5]

return {
    "advice": uniq,
    "source": {
        "predicted_label": predicted_label,
        "activity": act,
        "disease": dis,
        "nearest_bmi": nearest_bmi,
```

Hình 36



```
uniqu.append(a)
uniqu = uniqu[:5]

return {
    "advice": uniqu,
    "source": {
        "predicted_label": predicted_label,
        "activity": act,
        "disease": dis,
        "nearest_bmi": nearest_bmi,
        "bmi_advice": bmi_adv,
        "rules_used": rules_used
    }
}

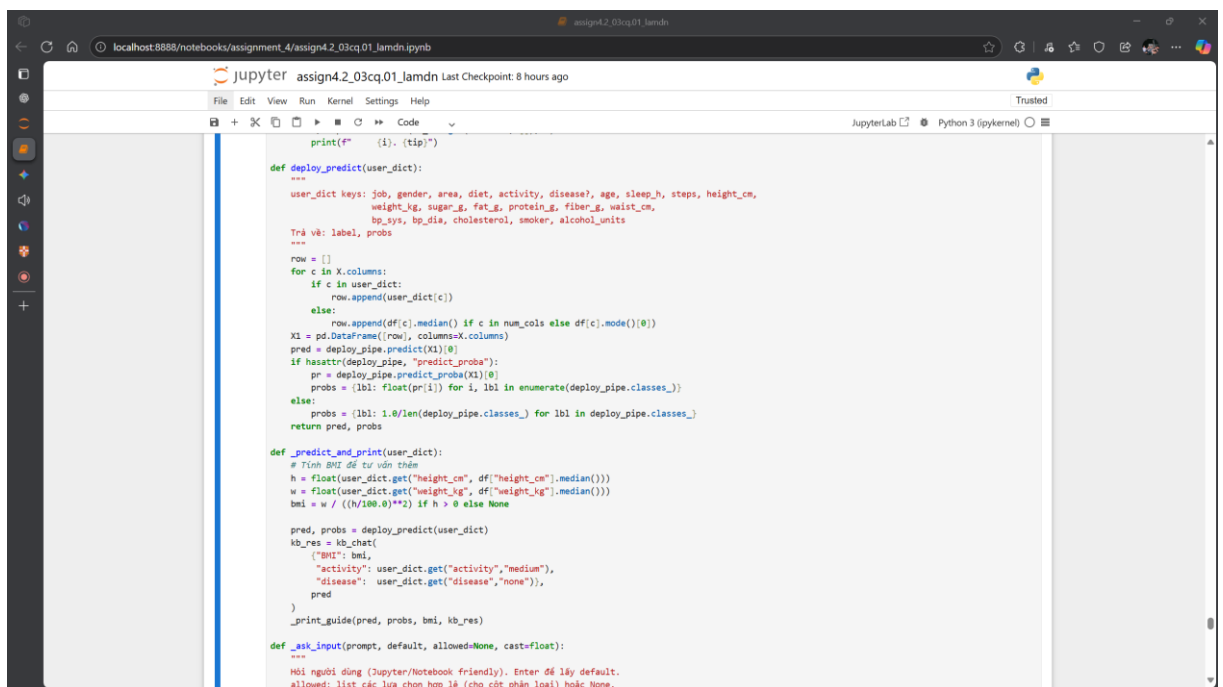
# =====
# (S) UI JUPYTER - CHẮC CHẮN HIỆN "GUIDE TỪ KB"
# =====
USE_WIDGETS = False # Để True nếu bạn đã cài ipynbwidgets

def _default_row():
    # Lấy giá trị mặc định: số = median, cat = mode
    row = {}
    for c in X.columns:
        if c in num_cols:
            row[c] = float(df[c].median())
        else:
            row[c] = str(df[c].mode()[0])
    return row

def _print_guide(pred, probs, bmi, kb_res):
    print("\n=== KẾT QUẢ DỰ ĐOÁN ===")
    print(f"Label: {pred}")
    if bmi is not None:
        print(f"BMI: {bmi:.2f}")
        print(f"Nhắc suất dự đoán:")
        for k in deploy_pipe.classes_:
            print(f" - {k.7s}: {probs.get(k, 0.0):.3f}")

    print("\n=== GUIDE TỪ KB ===")
    src = kb_res.get("source", {})
    print(f" Theo dự đoán: class='{src.get('predicted_label','?')}', activity='{src.get('activity','?')}', disease='{src.get('disease','none')}'")
    if src.get("nearest_bmi") is not None:
```

Hình 37



```
print(f" {i}. {tip}")

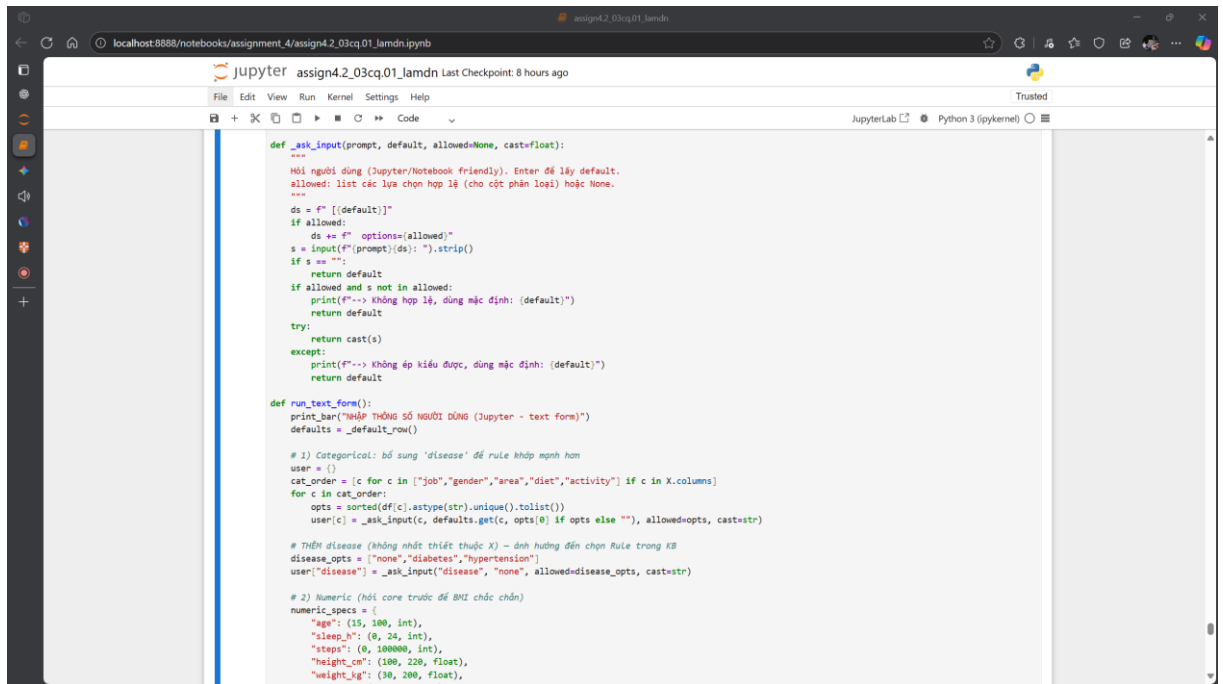
def deploy_predict(user_dict):
    """
    user_dict keys: job, gender, area, diet, activity, disease?, age, sleep_h, steps, height_cm,
    weight_kg, sugar_g, fat_g, protein_g, fiber_g, waist_cm,
    bp_sys, bp_dia, cholesterol, smoker, alcohol_units
    Trả về: label, probs
    """
    row = []
    for c in X.columns:
        if c in user_dict:
            row.append(user_dict[c])
        else:
            row.append(df[c].median() if c in num_cols else df[c].mode()[0])
    X1 = pd.DataFrame(row, columns=X.columns)
    pred = deploy_pipe.predict(X1)[0]
    if hasattr(deploy_pipe, "predict_proba"):
        pr = deploy_pipe.predict_proba(X1)[0]
        probs = {lbl: float(pr[i]) for i, lbl in enumerate(deploy_pipe.classes_)}
    else:
        probs = {lbl: 1.0/len(deploy_pipe.classes_) for lbl in deploy_pipe.classes_}
    return pred, probs

def _predict_and_print(user_dict):
    # Tính BMI để từ vốn thêm
    h = float(user_dict.get("height_cm", df["height_cm"].median()))
    w = float(user_dict.get("weight_kg", df["weight_kg"].median()))
    bmi = w / ((h/100.0)**2) if h > 0 else None

    pred, probs = deploy_predict(user_dict)
    kb_res = kb_chat(
        {"BMI": bmi,
         "activity": user_dict.get("activity", "medium"),
         "disease": user_dict.get("disease", "none")},
        pred
    )
    _print_guide(pred, probs, bmi, kb_res)

def _ask_input(prompt, default, allowed=None, cast=float):
    """
    Hỏi người dùng (Jupyter/Notebook friendly). Enter để lấy default.
    allowed: list các lựa chọn hợp lệ (cho cột phân loại) hoặc None.
    """
```

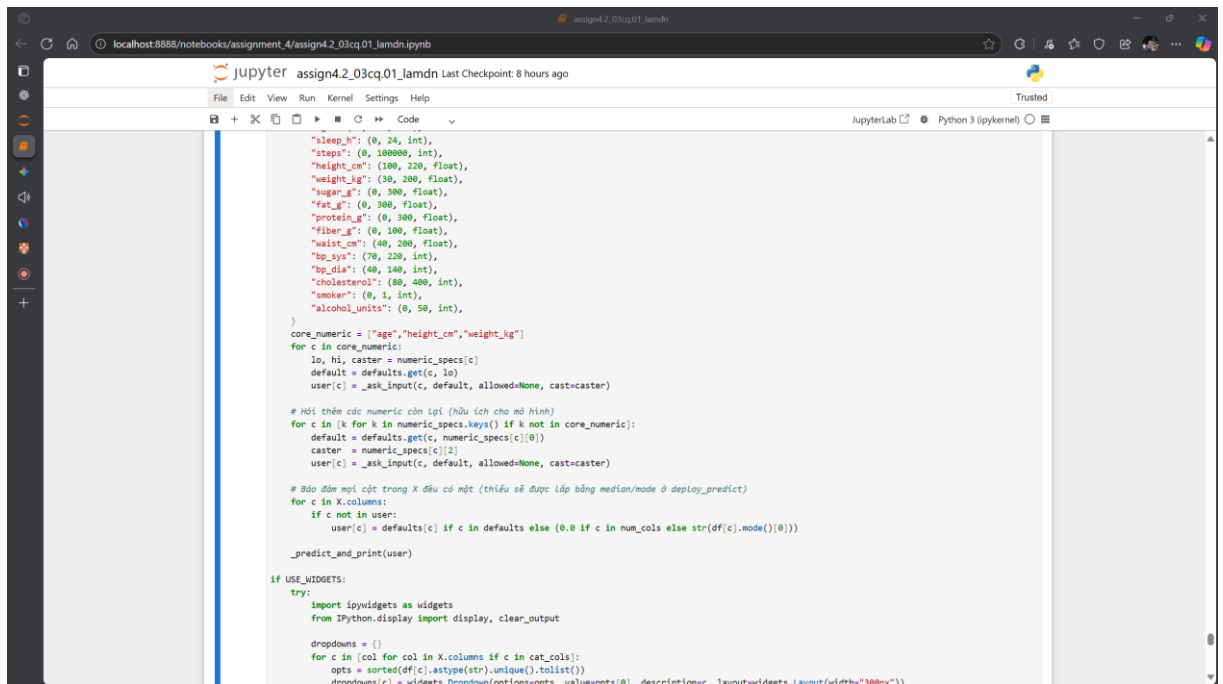
Hình 38



The screenshot shows a JupyterLab window titled "assign4.2_03cq.01_lamdn" with a last checkpoint of 8 hours ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations, running, and code execution. The main area contains a Python script with the following code:

```
def _ask_input(prompt, default, allowed=None, cast=float):  
    """  
    Hỏi người dùng (Jupyter/Notebook friendly). Enter để lấy default.  
    allowed: list các lựa chọn hợp lệ (cho cột phân loại) hoặc None.  
    """  
    ds = f" [{default}]"  
    if allowed:  
        ds += f" options={allowed}"  
    s = input(f"{prompt}{ds}: ").strip()  
    if s == "":  
        return default  
    if allowed and s not in allowed:  
        print(f"--> Không hợp lệ, dùng mặc định: {default}")  
        return default  
    try:  
        return cast(s)  
    except:  
        print(f"--> Không áp kiểu được, dùng mặc định: {default}")  
        return default  
  
def run_text_form():  
    print_bar("NHẬP THÔNG SỐ NGƯỜI DÙNG (Jupyter - text form)")  
    defaults = _default_row()  
  
    # 1) Categorical: bổ sung 'disease' để rule khớp mạnh hơn  
    user = {}  
    cat_order = [c for c in ["job", "gender", "area", "diet", "activity"] if c in X.columns]  
    for c in cat_order:  
        opts = sorted(df[c].astype(str).unique().tolist())  
        user[c] = _ask_input(c, defaults.get(c, opts[0]) if opts else "", allowed=opts, cast=str)  
  
    # Thêm disease (không nhất thiết thuộc X) - ảnh hưởng đến chọn Rule trong KB  
    disease_opts = ["none", "diabetes", "hypertension"]  
    user["disease"] = _ask_input("disease", "none", allowed=disease_opts, cast=str)  
  
    # 2) Numeric (hỏi core trước để BVI chắc chắn)  
    numeric_specs = {  
        "age": (15, 100, int),  
        "sleep_h": (0, 24, int),  
        "steps": (0, 100000, int),  
        "height_cm": (100, 220, float),  
        "weight_kg": (30, 200, float),  
        "sugar_g": (0, 300, float),  
        "fat_g": (0, 500, float),  
        "protein_g": (0, 300, float),  
        "fiber_g": (0, 100, float),  
        "waist_cm": (40, 200, float),  
        "bp_sys": (70, 220, int),  
        "bp_dia": (40, 140, int),  
        "cholesterol": (80, 400, int),  
        "smoker": (0, 1, int),  
        "alcohol_units": (0, 50, int),  
    }  
    core_numeric = ["age", "height_cm", "weight_kg"]  
    for c in core_numeric:  
        lo, hi, caster = numeric_specs[c]  
        default = defaults.get(c, lo)  
        user[c] = _ask_input(c, default, allowed=None, cast=caster)  
  
    # Hỏi thêm các numeric còn lại (nếu có cho mô hình)  
    for c in [k for k in numeric_specs.keys() if k not in core_numeric]:  
        default = defaults.get(c, numeric_specs[c][0])  
        caster = numeric_specs[c][2]  
        user[c] = _ask_input(c, default, allowed=None, cast=caster)  
  
    # Bảo đảm mọi cột trong X đều có một (thiếu sẽ được lấp bằng median/mode ở deploy_predict)  
    for c in X.columns:  
        if c not in user:  
            user[c] = defaults[c] if c in defaults else (0.0 if c in num_cols else str(df[c].mode()[0]))  
  
    _predict_and_print(user)  
  
if USE_WIDGETS:  
    try:  
        import ipywidgets as widgets  
        from IPython.display import display, clear_output  
  
        dropdowns = {}  
        for c in [col for col in X.columns if c in cat_cols]:  
            opts = sorted(df[c].astype(str).unique().tolist())  
            dropdowns[c] = widgets.Dropdown(options=opts, value=opts[0], description=c, layout=widgets.Layout(width="300px"))
```

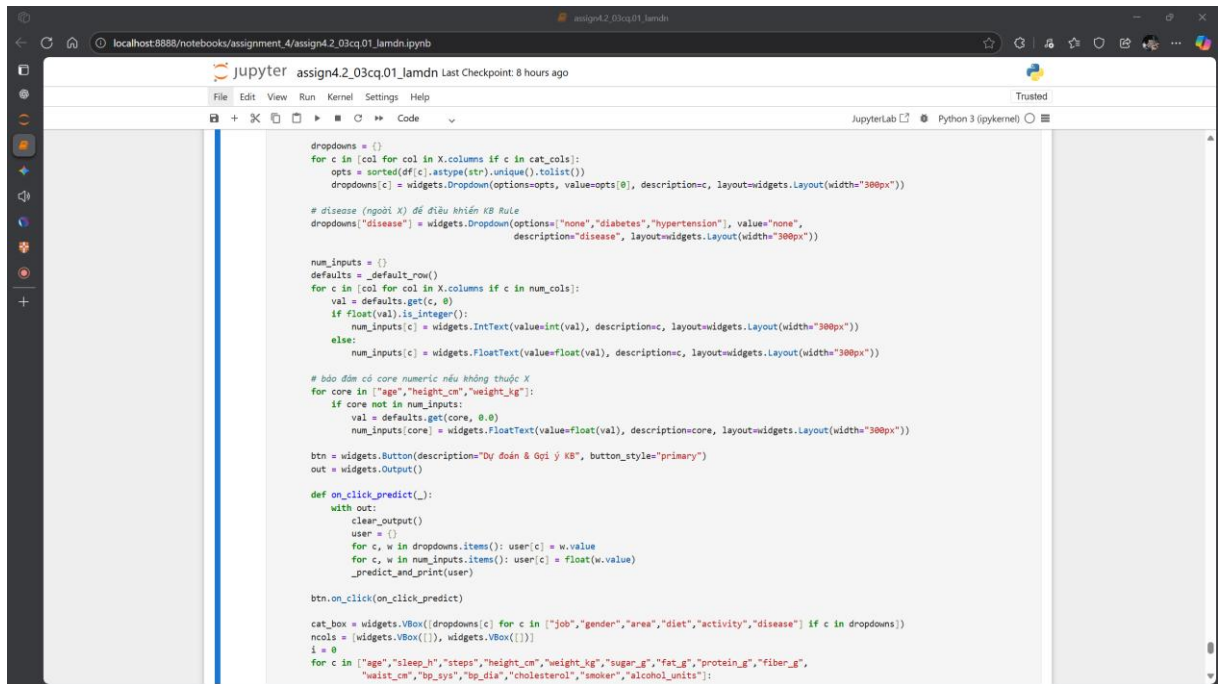
Hình 39



The screenshot shows a JupyterLab window titled "assign4.2_03cq.01_lamdn" with a last checkpoint of 8 hours ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations, running, and code execution. The main area contains a Python script with the following code:

```
"sleep_h": (0, 24, int),  
"steps": (0, 100000, int),  
"height_cm": (100, 220, float),  
"weight_kg": (30, 200, float),  
"sugar_g": (0, 300, float),  
"fat_g": (0, 500, float),  
"protein_g": (0, 300, float),  
"fiber_g": (0, 100, float),  
"waist_cm": (40, 200, float),  
"bp_sys": (70, 220, int),  
"bp_dia": (40, 140, int),  
"cholesterol": (80, 400, int),  
"smoker": (0, 1, int),  
"alcohol_units": (0, 50, int),  
}  
core_numeric = ["age", "height_cm", "weight_kg"]  
for c in core_numeric:  
    lo, hi, caster = numeric_specs[c]  
    default = defaults.get(c, lo)  
    user[c] = _ask_input(c, default, allowed=None, cast=caster)  
  
# Hỏi thêm các numeric còn lại (nếu có cho mô hình)  
for c in [k for k in numeric_specs.keys() if k not in core_numeric]:  
    default = defaults.get(c, numeric_specs[c][0])  
    caster = numeric_specs[c][2]  
    user[c] = _ask_input(c, default, allowed=None, cast=caster)  
  
# Bảo đảm mọi cột trong X đều có một (thiếu sẽ được lấp bằng median/mode ở deploy_predict)  
for c in X.columns:  
    if c not in user:  
        user[c] = defaults[c] if c in defaults else (0.0 if c in num_cols else str(df[c].mode()[0]))  
  
_predict_and_print(user)  
  
if USE_WIDGETS:  
    try:  
        import ipywidgets as widgets  
        from IPython.display import display, clear_output  
  
        dropdowns = {}  
        for c in [col for col in X.columns if c in cat_cols]:  
            opts = sorted(df[c].astype(str).unique().tolist())  
            dropdowns[c] = widgets.Dropdown(options=opts, value=opts[0], description=c, layout=widgets.Layout(width="300px"))
```

Hình 40



```
dropdowns = {}
for c in [col for col in X.columns if c in cat_cols]:
    opts = sorted(df[c].astype(str).unique().tolist())
    dropdowns[c] = widgets.Dropdown(options=opts, value=opts[0], description=c, layout=widgets.Layout(width="300px"))

# disease (nguồn X) để điều khiển KB Rule
dropdowns["disease"] = widgets.Dropdown(options=["none", "diabetes", "hypertension"], value="none",
                                         description="disease", layout=widgets.Layout(width="300px"))

num_inputs = {}
defaults = _default_row()
for c in [col for col in X.columns if c in num_cols]:
    val = defaults.get(c, 0)
    if float(val).is_integer():
        num_inputs[c] = widgets.IntText(value=int(val), description=c, layout=widgets.Layout(width="300px"))
    else:
        num_inputs[c] = widgets.FloatText(value=float(val), description=c, layout=widgets.Layout(width="300px"))

# bảo đảm có core numeric nếu không thuộc X
for core in ["age", "height_cm", "weight_kg"]:
    if core not in num_inputs:
        val = defaults.get(core, 0.0)
        num_inputs[core] = widgets.FloatText(value=float(val), description=core, layout=widgets.Layout(width="300px"))

btn = widgets.Button(description="Dự đoán & Gợi ý KB", button_style="primary")
out = widgets.Output()

def on_click_predict(_):
    with out:
        clear_output()
        user = {}
        for c, w in dropdowns.items(): user[c] = w.value
        for c, w in num_inputs.items(): user[c] = float(w.value)
        _predict_and_print(user)

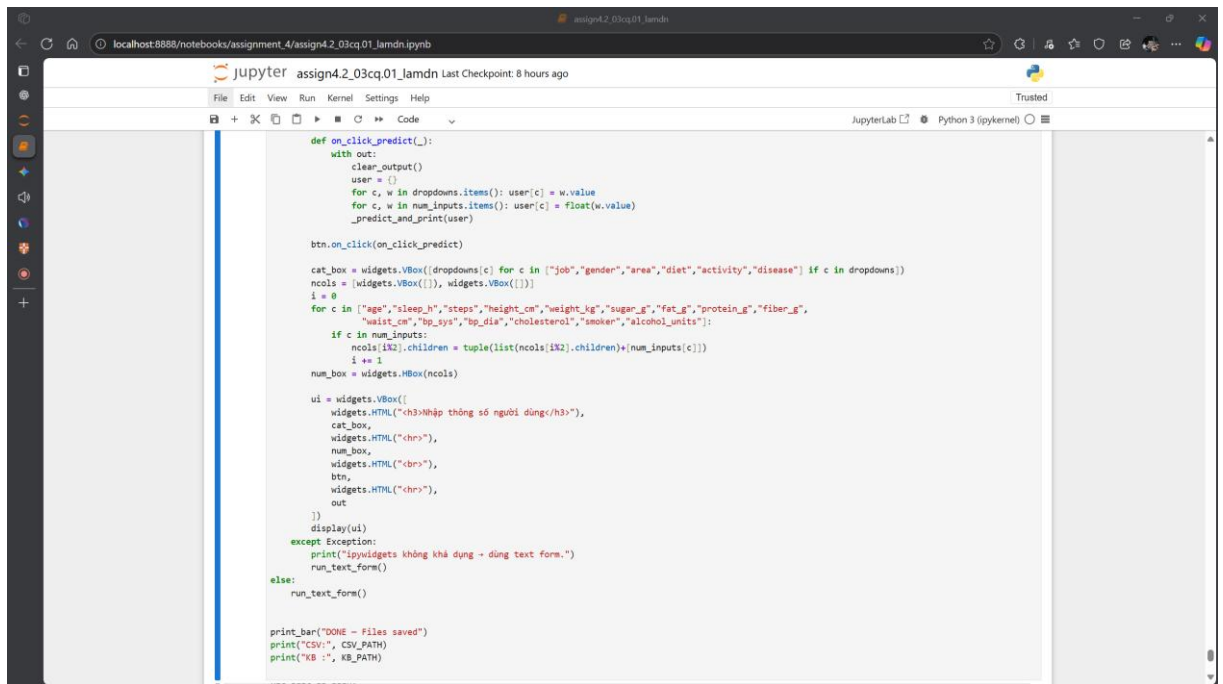
btn.on_click(on_click_predict)

cat_box = widgets.VBox([dropdowns[c] for c in ["job", "gender", "area", "diet", "activity", "disease"] if c in dropdowns])
ncols = [widgets.VBox([ ]), widgets.VBox([ ])]
i = 0
for c in ["age", "sleep_h", "steps", "height_cm", "weight_kg", "sugar_g", "fat_g", "protein_g", "fiber_g",
          "waist_cm", "bp_sys", "bp_dia", "cholesterol", "smoker", "alcohol_units"]:
    if c in num_inputs:
        ncols[i%2].children = tuple(list(ncols[i%2].children)+[num_inputs[c]])
        i += 1
num_box = widgets.HBox(ncols)

ui = widgets.VBox([
    widgets.HTML("<h3>Nhập thông số người dùng</h3>"),
    cat_box,
    widgets.HTML("<br>"),
    num_box,
    widgets.HTML("<br>"),
    btn,
    widgets.HTML("<br>"),
    out
])
display(ui)
except Exception:
    print("ipywidgets không khả dụng - dùng text form.")
    run_text_form()
else:
    run_text_form()

print_bar("DONE - Files saved")
print("CSV:", CSV_PATH)
print("KB :", KB_PATH)
```

Hình 41



```
def on_click_predict(_):
    with out:
        clear_output()
        user = {}
        for c, w in dropdowns.items(): user[c] = w.value
        for c, w in num_inputs.items(): user[c] = float(w.value)
        _predict_and_print(user)

btn.on_click(on_click_predict)

cat_box = widgets.VBox([dropdowns[c] for c in ["job", "gender", "area", "diet", "activity", "disease"] if c in dropdowns])
ncols = [widgets.VBox([ ]), widgets.VBox([ ])]
i = 0
for c in ["age", "sleep_h", "steps", "height_cm", "weight_kg", "sugar_g", "fat_g", "protein_g", "fiber_g",
          "waist_cm", "bp_sys", "bp_dia", "cholesterol", "smoker", "alcohol_units"]:
    if c in num_inputs:
        ncols[i%2].children = tuple(list(ncols[i%2].children)+[num_inputs[c]])
        i += 1
num_box = widgets.HBox(ncols)

ui = widgets.VBox([
    widgets.HTML("<h3>Nhập thông số người dùng</h3>"),
    cat_box,
    widgets.HTML("<br>"),
    num_box,
    widgets.HTML("<br>"),
    btn,
    widgets.HTML("<br>"),
    out
])
display(ui)
except Exception:
    print("ipywidgets không khả dụng - dùng text form.")
    run_text_form()
else:
    run_text_form()

print_bar("DONE - Files saved")
print("CSV:", CSV_PATH)
print("KB :", KB_PATH)
```

Hình 42

```
===== DEPLOY MODEL = LogReg =====
===== KB READY =====
KB file: C:\Users\Admin\Desktop\PTIT\V4_T1\IntSys\assignment_4\kb_healthGuide_new.json | size MB: 3.22
===== NHẬP THÔNG SỐ NGƯỜI DÙNG (Jupyter - text form) =====
Job [nurse] options=['athlete', 'chef', 'driver', 'engineer', 'freelance', 'nurse', 'office', 'sales', 'student', 'teacher']:
gender [male] options=['female', 'male']:
area [rural] options=['rural', 'suburban', 'urban']:
diet [vegetarian] options=['balanced', 'dash', 'high_carb', 'high_fat', 'keto', 'mediterranean', 'vegetarian']:
activity [high] options=['high', 'low', 'medium']:
disease [none] options=['none', 'diabetes', 'hypertension']:
age [42.0]:
height_cm [165.5]:
weight_kg [68.4]:
sleep_h [7.0]:
steps [10625.5]:
sugar_g [59.6]:
fat_g [66.1]:
protein_g [65.9]:
fiber_g [20.0]:
waist_cm [75.0]:
bp_sys [120.0]:
bp_dia [77.0]:
cholesterol [185.0]:
smoker [1.0]:
alcohol_units [10.0]:

=== KẾT QUẢ DỰ ĐOÁN ===
Label: normal
BMI: 22.05

Xác suất dự đoán:
- normal : 1.000
- over : 0.000
- under : 0.000

=== GUIDE TỪ KB ===
• Theo dự đoán: class='normal', activity='high', disease='none'
• BMI gần nhất trong KB: 22.1
• Rule áp dụng: RL005950, RL005951, RL005952
• Gợi ý (top):
  1. Duy trì năng lượng
  2. Ăn đa dạng
  3. Cardio 150 phút/tuần
  4. Tham vấn dinh dưỡng
```

Hình 43

```
activity [high] options=['high', 'low', 'medium']:
disease [none] options=['none', 'diabetes', 'hypertension']:
age [42.0]:
height_cm [165.5]:
weight_kg [68.4]:
sleep_h [7.0]:
steps [10625.5]:
sugar_g [59.6]:
fat_g [66.1]:
protein_g [65.9]:
fiber_g [20.0]:
waist_cm [75.0]:
bp_sys [120.0]:
bp_dia [77.0]:
cholesterol [185.0]:
smoker [1.0]:
alcohol_units [10.0]:

=== KẾT QUẢ DỰ ĐOÁN ===
Label: normal
BMI: 22.05

Xác suất dự đoán:
- normal : 1.000
- over : 0.000
- under : 0.000

=== GUIDE TỪ KB ===
• Theo dự đoán: class='normal', activity='high', disease='none'
• BMI gần nhất trong KB: 22.1
• Rule áp dụng: RL005950, RL005951, RL005952
• Gợi ý (top):
  1. Duy trì năng lượng
  2. Ăn đa dạng
  3. Cardio 150 phút/tuần
  4. Tham vấn dinh dưỡng
  5. Giảm carb tinh luyện

===== DONE - Files saved =====
CSV: C:\Users\Admin\Desktop\PTIT\V4_T1\IntSys\assignment_4\data_4-2_new.csv
KB : C:\Users\Admin\Desktop\PTIT\V4_T1\IntSys\assignment_4\kb_healthGuide_new.json

[ ]:
```

Hình 44

→ kb_chat: Lấy gợi ý từ knowledge base

- Mục tiêu: trả về danh sách advice và source dựa trên BMI gần nhất, Rules,
- Quy trình: lấy BMI, activity, disease từ context và predicted_label từ model rồi lấy BMI gần nhất trong bảng kb["BMI_class"], tìm entry có BMI gần nhất rồi lấy ra advice của entry đó. Duyệt kb["rules"] lấy ra 3 rule đầu tiên thỏa mãn điều kiện

➔ Deploy_predict():

- Mục tiêu: Dùng pipeline tốt nhất đã lưu để dự đoán lớp và xác suất

➔ _predict_and_print():

- Mục tiêu: Kết nối dự đoán của model với tư vấn từ KB

➔ _ask_input()

- Mục tiêu: Cho phép nhập liệu nhau trong ô lệnh Jupyter Notebook